



Red Hat OpenShift AI Self-Managed 2.25

Installing and uninstalling OpenShift AI Self-Managed in a disconnected environment

Install and uninstall OpenShift AI Self-Managed in a disconnected environment

Red Hat OpenShift AI Self-Managed 2.25 Installing and uninstalling OpenShift AI Self-Managed in a disconnected environment

Install and uninstall OpenShift AI Self-Managed in a disconnected environment

Legal Notice

Copyright © 2025 Red Hat, Inc.

The text of and illustrations in this document are licensed by Red Hat under a Creative Commons Attribution–Share Alike 3.0 Unported license ("CC-BY-SA"). An explanation of CC-BY-SA is available at

<http://creativecommons.org/licenses/by-sa/3.0/>

. In accordance with CC-BY-SA, if you distribute this document or an adaptation of it, you must provide the URL for the original version.

Red Hat, as the licensor of this document, waives the right to enforce, and agrees not to assert, Section 4d of CC-BY-SA to the fullest extent permitted by applicable law.

Red Hat, Red Hat Enterprise Linux, the Shadowman logo, the Red Hat logo, JBoss, OpenShift, Fedora, the Infinity logo, and RHCE are trademarks of Red Hat, Inc., registered in the United States and other countries.

Linux[®] is the registered trademark of Linus Torvalds in the United States and other countries.

Java[®] is a registered trademark of Oracle and/or its affiliates.

XFS[®] is a trademark of Silicon Graphics International Corp. or its subsidiaries in the United States and/or other countries.

MySQL[®] is a registered trademark of MySQL AB in the United States, the European Union and other countries.

Node.js[®] is an official trademark of Joyent. Red Hat is not formally related to or endorsed by the official Joyent Node.js open source or commercial project.

The OpenStack[®] Word Mark and OpenStack logo are either registered trademarks/service marks or trademarks/service marks of the OpenStack Foundation, in the United States and other countries and are used with the OpenStack Foundation's permission. We are not affiliated with, endorsed or sponsored by the OpenStack Foundation, or the OpenStack community.

All other trademarks are the property of their respective owners.

Abstract

Install and uninstall OpenShift AI Self-Managed on your OpenShift cluster in a disconnected environment.

Table of Contents

PREFACE	4
CHAPTER 1. ARCHITECTURE OF OPENSIFT AI SELF-MANAGED	5
CHAPTER 2. UNDERSTANDING UPDATE CHANNELS	7
CHAPTER 3. DEPLOYING OPENSIFT AI IN A DISCONNECTED ENVIRONMENT	9
3.1. REQUIREMENTS FOR OPENSIFT AI SELF-MANAGED	9
3.2. MIRRORING IMAGES TO A PRIVATE REGISTRY FOR A DISCONNECTED INSTALLATION	12
3.3. CONFIGURING CUSTOM NAMESPACES	16
3.4. INSTALLING THE RED HAT OPENSIFT AI OPERATOR	18
3.4.1. Installing the Red Hat OpenShift AI Operator by using the CLI	18
3.4.2. Installing the Red Hat OpenShift AI Operator by using the web console	21
3.5. INSTALLING AND MANAGING RED HAT OPENSIFT AI COMPONENTS	23
3.5.1. Installing Red Hat OpenShift AI components by using the CLI	23
3.5.2. Installing Red Hat OpenShift AI components by using the web console	26
3.5.3. Updating the installation status of Red Hat OpenShift AI components by using the web console	30
3.5.4. Viewing installed OpenShift AI components	33
CHAPTER 4. CONFIGURING PIPELINES WITH YOUR OWN ARGO WORKFLOWS INSTANCE	34
CHAPTER 5. INSTALLING THE DISTRIBUTED WORKLOADS COMPONENTS	36
CHAPTER 6. INSTALLING THE SINGLE-MODEL SERVING PLATFORM	39
6.1. ABOUT THE SINGLE-MODEL SERVING PLATFORM	39
6.2. CONFIGURING AUTOMATED INSTALLATION OF KSERVE	39
6.3. MANUALLY INSTALLING KSERVE	45
6.3.1. Installing KServe dependencies	45
6.3.2. Creating an OpenShift Service Mesh instance	45
6.3.3. Creating a Knative Serving instance	47
6.3.4. Creating secure gateways for Knative Serving	51
6.3.5. Installing KServe	55
6.3.6. Configuring persistent volume claims (PVC) on KServe	56
6.3.7. Disabling KServe dependencies	57
6.4. ADDING AN AUTHORIZATION PROVIDER FOR THE SINGLE-MODEL SERVING PLATFORM	58
6.4.1. Manually adding an authorization provider	58
6.4.2. Installing the Red Hat Authorino Operator	59
6.4.3. Creating an Authorino instance	59
6.4.4. Configuring an OpenShift Service Mesh instance to use Authorino	61
6.4.5. Configuring authorization for KServe	63
CHAPTER 7. INSTALLING THE MULTI-MODEL SERVING PLATFORM	66
CHAPTER 8. ACCESSING THE DASHBOARD	67
CHAPTER 9. ENABLING ACCELERATORS	68
CHAPTER 10. WORKING WITH CERTIFICATES	70
10.1. UNDERSTANDING HOW OPENSIFT AI HANDLES CERTIFICATES	70
10.2. ADDING CERTIFICATES	71
10.3. ADDING CERTIFICATES TO A CLUSTER-WIDE CA BUNDLE	72
10.4. ADDING CERTIFICATES TO A CUSTOM CA BUNDLE	73
10.5. USING SELF-SIGNED CERTIFICATES WITH OPENSIFT AI COMPONENTS	74
10.5.1. Accessing S3-compatible object storage with self-signed certificates	74

10.5.2. Configuring a certificate for data science pipelines	75
10.5.3. Configuring a certificate for workbenches	77
10.5.4. Using the cluster-wide CA bundle for the single-model serving platform	77
10.6. MANAGING CERTIFICATES WITHOUT THE RED HAT OPENSIFT AI OPERATOR	78
10.7. REMOVING THE CA BUNDLE	80
10.7.1. Removing the CA bundle from all namespaces	80
10.7.2. Removing the CA bundle from a single namespace	81
CHAPTER 11. VIEWING LOGS AND AUDIT RECORDS	82
11.1. CONFIGURING THE OPENSIFT AI OPERATOR LOGGER	82
11.1.1. Viewing the OpenShift AI Operator logs	83
11.2. VIEWING AUDIT RECORDS	83
CHAPTER 12. TROUBLESHOOTING COMMON INSTALLATION PROBLEMS	86
12.1. THE RED HAT OPENSIFT AI OPERATOR CANNOT BE RETRIEVED FROM THE IMAGE REGISTRY	86
12.2. OPENSIFT AI DOES NOT INSTALL ON UNSUPPORTED INFRASTRUCTURE	86
12.3. THE CREATION OF THE OPENSIFT AI CUSTOM RESOURCE (CR) FAILS	87
12.4. THE CREATION OF THE OPENSIFT AI NOTEBOOKS CUSTOM RESOURCE (CR) FAILS	87
12.5. THE OPENSIFT AI DASHBOARD IS NOT ACCESSIBLE	88
12.6. REINSTALLING OPENSIFT AI FAILS WITH AN ERROR	88
12.7. THE DEDICATED-ADMINS ROLE-BASED ACCESS CONTROL (RBAC) POLICY CANNOT BE CREATED	89
12.8. THE PAGERDUTY SECRET DOES NOT GET CREATED	90
12.9. THE SMTP SECRET DOES NOT EXIST	90
12.10. THE ODH PARAMETER SECRET DOES NOT GET CREATED	91
CHAPTER 13. UNINSTALLING RED HAT OPENSIFT AI SELF-MANAGED	92
13.1. UNDERSTANDING THE UNINSTALLATION PROCESS	92
13.2. UNINSTALLING OPENSIFT AI SELF-MANAGED BY USING THE CLI	93

PREFACE

Learn how to use both the OpenShift CLI (**oc**) and web console to install Red Hat OpenShift AI Self-Managed on your OpenShift cluster in a disconnected environment. To uninstall the product, learn how to use the recommended command-line interface (CLI) method.



NOTE

Red Hat does not support installing more than one instance of OpenShift AI on your cluster.

Red Hat does not support installing the Red Hat OpenShift AI Operator on the same cluster as the Red Hat OpenShift AI Add-on.

CHAPTER 1. ARCHITECTURE OF OPENSIFT AI SELF-MANAGED

Red Hat OpenShift AI Self-Managed is an Operator that is available in a self-managed environment, such as Red Hat OpenShift Container Platform, or in Red Hat-managed cloud environments such as Red Hat OpenShift Dedicated (with a Customer Cloud Subscription for AWS or GCP), Red Hat OpenShift Service on Amazon Web Services (ROSA classic or ROSA HCP), or Microsoft Azure Red Hat OpenShift.

OpenShift AI integrates the following components and services:

- At the service layer:

OpenShift AI dashboard

A customer-facing dashboard that shows available and installed applications for the OpenShift AI environment as well as learning resources such as tutorials, quick starts, and documentation. Administrative users can access functionality to manage users, clusters, workbench images, accelerator profiles, hardware profiles, and model-serving runtimes. Data scientists can use the dashboard to create projects to organize their data science work.



IMPORTANT

By default, hardware profiles are hidden in the dashboard navigation menu and user interface, while accelerator profiles remain visible. In addition, user interface components associated with the deprecated accelerator profiles functionality are still displayed. To show the **Settings → Hardware profiles** option in the dashboard navigation menu, and the user interface components associated with hardware profiles, set the **disableHardwareProfiles** value to **false** in the **OdhDashboardConfig** custom resource (CR) in OpenShift. For more information about setting dashboard configuration options, see [Customizing the dashboard](#).

Model serving

Data scientists can deploy trained machine-learning models to serve intelligent applications in production. After deployment, applications can send requests to the model using its deployed API endpoint.

Data science pipelines

Data scientists can build portable machine learning (ML) workflows with data science pipelines 2.0, using Docker containers. With data science pipelines, data scientists can automate workflows as they develop their data science models.

Jupyter (self-managed)

A self-managed application that allows data scientists to configure a basic standalone workbench and develop machine learning models in JupyterLab.

Distributed workloads

Data scientists can use multiple nodes in parallel to train machine-learning models or process data more quickly. This approach significantly reduces the task completion time, and enables the use of larger datasets and more complex models.

Retrieval-Augmented Generation (RAG)

Data scientists and AI engineers can leverage Retrieval-Augmented Generation (RAG) capabilities provided by the integrated Llama Stack Operator. By combining large language model inference, semantic retrieval, and vector database storage, data scientists and AI

engineers can obtain tailored, accurate, and verifiable answers to complex queries based on their own datasets within a data science project.

- At the management layer:

The Red Hat OpenShift AI Operator

A meta-operator that deploys and maintains all components and sub-operators that are part of OpenShift AI.

When you install the Red Hat OpenShift AI Operator in the OpenShift cluster using the predefined projects, the following new projects are created:

- The **redhat-ods-operator** project contains the Red Hat OpenShift AI Operator.
- The **redhat-ods-applications** project includes the dashboard and other required components of OpenShift AI.
- The **rhods-notebooks** project is where basic workbenches are deployed by default.

You can specify custom projects if needed. You or your data scientists must also create additional projects for the applications that will use your machine learning models.

Do not install independent software vendor (ISV) applications in namespaces associated with OpenShift AI.

CHAPTER 2. UNDERSTANDING UPDATE CHANNELS

You can use update channels to specify which Red Hat OpenShift AI minor version you intend to update your Operator to. Update channels also allow you to choose the timing and level of support your updates have through the **fast**, **stable**, **stable-x.y**, **eus-x.y**, and **alpha** channel options.

The subscription of an installed Operator specifies the update channel, which is used to track and receive updates for the Operator. You can change the update channel to start tracking and receiving updates from a newer channel. For more information about the release frequency and the lifecycle associated with each of the available update channels, see the [Red Hat OpenShift AI Self-Managed Life Cycle](#) Knowledgebase article.

Channel	Support	Release frequency	Recommended environment
fast	One month of full support	Every month	Production environments with access to the latest product features. Select this streaming channel with automatic updates to avoid manually upgrading every month.
stable	Three months of full support	Every three months	Production environments with stability prioritized over new feature availability. Select this streaming channel with automatic updates to access the latest stable release and avoid manually upgrading.
stable-x.y	Seven months of full support	Every three months	Production environments with stability prioritized over new feature availability. Select numbered stable channels (such as stable-2.10) to plan and upgrade to the next stable release while keeping your deployment under full support.
eus-x.y	Seven months of full support followed by Extended Update Support for eleven months	Every nine months	Enterprise-grade environments that cannot upgrade within a seven month window. Select this streaming channel if you prioritize stability over new feature availability.

Channel	Support	Release frequency	Recommended environment
alpha	One month of full support	Every month	Development environments with early-access features that might not be functionally complete. Select this channel to use early-access features to test functionality and provide feedback during the development process. Early-access features are not supported with Red Hat production service level agreements (SLAs). For more information about the support scope of Red Hat Technology Preview features, see Technology Preview Features Support Scope. For more information about the support scope of Red Hat Developer Preview features, see Developer Preview Features Support Scope.



NOTE

The **embedded** and **beta** channels are legacy channels that will be removed in a future release. Do not select the **embedded** or **beta** channels for a new Operator installation.

CHAPTER 3. DEPLOYING OPENSIFT AI IN A DISCONNECTED ENVIRONMENT

Read this section to understand how to deploy Red Hat OpenShift AI as a development and testing environment for data scientists in a disconnected environment. Disconnected clusters are on a restricted network, typically behind a firewall. In this case, clusters cannot access the remote registries where Red Hat provided OperatorHub sources reside. Instead, the Red Hat OpenShift AI Operator can be deployed to a disconnected environment using a private registry to mirror the images.

Installing OpenShift AI in a disconnected environment involves the following high-level tasks:

1. Confirm that your OpenShift cluster meets all requirements. See [Requirements for OpenShift AI Self-Managed](#).
2. Mirror images to a private registry. See [Mirroring images to a private registry for a disconnected installation](#).
3. Install the Red Hat OpenShift AI Operator. See [Installing the Red Hat OpenShift AI Operator](#).
4. Install OpenShift AI components. See [Installing and managing Red Hat OpenShift AI components](#).
5. Configure user and administrator groups to provide user access to OpenShift AI. See [Adding users to OpenShift AI user groups](#).
6. Provide your users with the URL for the OpenShift cluster on which you deployed OpenShift AI. See [Accessing the OpenShift AI dashboard](#).
7. Optionally, configure and enable your accelerators in OpenShift AI to ensure that your data scientists can use compute-heavy workloads in their models. See [Enabling accelerators](#).

3.1. REQUIREMENTS FOR OPENSIFT AI SELF-MANAGED

You must meet the following requirements before you can install Red Hat OpenShift AI on your Red Hat OpenShift cluster in a disconnected environment:

Product subscriptions

- You must have a subscription for Red Hat OpenShift AI Self-Managed. Contact your Red Hat account manager to purchase new subscriptions. If you do not yet have an account manager, complete the form at <https://www.redhat.com/en/contact> to request one.

Cluster administrator access to your OpenShift cluster

- You must have an OpenShift cluster with cluster administrator access. Use an existing cluster or create a cluster by following the OpenShift Container Platform documentation: [Installing a cluster in a disconnected environment](#).
- After you install a cluster, configure the Cluster Samples Operator by following the OpenShift Container Platform documentation: [Configuring Samples Operator for a restricted cluster](#).
- Your cluster must have at least 2 worker nodes with at least 8 CPUs and 32 GiB RAM available for OpenShift AI to use when you install the Operator. To ensure that OpenShift AI is usable, additional cluster resources are required beyond the minimum requirements.

- To use OpenShift AI on single node OpenShift, the node has to have at least 32 CPUs and 128 GiB RAM.
- Your cluster is configured with a default storage class that can be dynamically provisioned. Confirm that a default storage class is configured by running the **oc get storageclass** command. If no storage classes are noted with **(default)** beside the name, follow the OpenShift Container Platform documentation to configure a default storage class: [Changing the default storage class](#). For more information about dynamic provisioning, see [Dynamic provisioning](#).
- Open Data Hub must not be installed on the cluster.

For more information about managing the machines that make up an OpenShift cluster, see [Overview of machine management](#).

An identity provider configured for OpenShift

- Red Hat OpenShift AI uses the same authentication systems as Red Hat OpenShift Container Platform. See [Understanding identity provider configuration](#) for more information on configuring identity providers.
- Access to the cluster as a user with the **cluster-admin** role; the **kubeadmin** user is not allowed. To assign **cluster-admin** privileges to a user, follow the steps in the relevant OpenShift documentation:
 - OpenShift Container Platform: [Creating a cluster admin](#)
 - OpenShift Dedicated: [Managing OpenShift Dedicated administrators](#)
 - ROSA: [Creating a cluster administrator user for quick cluster access](#)

Internet access on the mirroring machine

- Along with Internet access, the following domains must be accessible to mirror images required for the OpenShift AI Self-Managed installation:
 - **cdn.redhat.com**
 - **subscription.rhn.redhat.com**
 - **registry.access.redhat.com**
 - **registry.redhat.io**
 - **quay.io**
- For environments that build or customize CUDA-based images using NVIDIA's base images, or that directly pull artifacts from the NVIDIA NGC catalog, the following domains must also be accessible:
 - **ngc.download.nvidia.cn**
 - **developer.download.nvidia.com**



NOTE

Access to these NVIDIA domains is not required for standard OpenShift AI Self-Managed installations. The CUDA-based container images used by OpenShift AI are prebuilt and hosted on Red Hat's registry at **registry.redhat.io**.

Create custom namespaces

- By default, OpenShift AI uses predefined namespaces, but you can define custom namespaces for the operator, applications, and workbenches if needed. Namespaces created by OpenShift AI typically include **openshift** or **redhat** in their name. Do not rename these system namespaces because they are required for OpenShift AI to function properly. If you are using custom namespaces, before installing the OpenShift AI Operator, you must have created and labeled them as required.
- Before you can execute a pipeline in a disconnected environment, you must upload the images to your private registry. For more information, see [Mirroring images to run pipelines in a restricted environment](#).
- You can store your pipeline artifacts in an S3-compatible object storage bucket so that you do not consume local storage. To do this, you must first configure write access to your S3 bucket on your storage account.
- If you are installing OpenShift AI on a cluster running in FIPS mode, any custom container images for data science pipelines must be based on UBI 9 or RHEL 9. This ensures compatibility with FIPS-approved pipeline components and prevents errors related to mismatched OpenSSL or GNU C Library (glibc) versions.

Install KServe dependencies

- To support the KServe component, which is used by the single-model serving platform to serve large models, you must also install Operators for Red Hat OpenShift Serverless and Red Hat OpenShift Service Mesh and perform additional configuration. For more information, see [About the single-model serving platform](#).
- If you want to add an authorization provider for the single-model serving platform, you must install the **Red Hat - Authorino** Operator. For information, see [Adding an authorization provider for the single-model serving platform](#).

Install RAG dependencies

If you plan to deploy Retrieval-Augmented Generation (RAG) workloads by using Llama Stack, you must meet the following requirements:

- You have GPU-enabled nodes available on your cluster and you have installed the Node Feature Discovery Operator and NVIDIA GPU Operator. For more information, see [Installing the Node Feature Discovery Operator](#) and [Enabling NVIDIA GPUs](#).
- You have access to storage for your model artifacts.
- You have met the KServe installation prerequisites.

Access to object storage

- Components of OpenShift AI require or can use S3-compatible object storage such as AWS S3, MinIO, Ceph, or IBM Cloud Storage. An object store is a data storage mechanism that enables

users to access their data either as an object or as a file. The S3 API is the recognized standard for HTTP-based access to object storage services.

- The object storage must be accessible to your OpenShift cluster. Deploy the object storage on the same disconnected network as your cluster.
- Object storage is required for the following components:
 - Single- or multi-model serving platforms, to deploy stored models. See [Deploying models on the single-model serving platform](#) or [Deploying a model by using the multi-model serving platform](#).
 - Data science pipelines, to store artifacts, logs, and intermediate results. See [Configuring a pipeline server](#) and [About pipeline logs](#).
- Object storage can be used by the following components:
 - Workbenches, to access large datasets. See [Adding a connection to your data science project](#).
 - Distributed workloads, to pull input data from and push results to. See [Running distributed data science workloads from data science pipelines](#).
 - Code executed inside a pipeline. For example, to store the resulting model in object storage. See [Overview of pipelines in Jupyterlab](#).

3.2. MIRRORING IMAGES TO A PRIVATE REGISTRY FOR A DISCONNECTED INSTALLATION

You can install the Red Hat OpenShift AI Operator to your OpenShift cluster in a disconnected environment by mirroring the required container images to a private container registry. After mirroring the images to a container registry, you can install Red Hat OpenShift AI Operator by using OperatorHub.

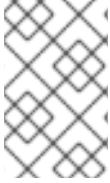
You can use the *mirror registry for Red Hat OpenShift*, a small-scale container registry, as a target for mirroring the required container images for OpenShift AI in a disconnected environment. Using the mirror registry for Red Hat OpenShift is optional if another container registry is already available in your installation environment.

Prerequisites

- You have cluster administrator access to a running OpenShift Container Platform cluster, version 4.16 or greater.
- You have credentials for Red Hat OpenShift Cluster Manager (<https://console.redhat.com/openshift/>).
- Your mirroring machine is running Linux, has 100 GB of space available, and has access to the Internet so that it can obtain the images to populate the mirror repository.
- You have installed the OpenShift CLI (**oc**).
- You have reviewed the component requirements and identified all operators you must mirror in addition to the Red Hat OpenShift AI Operator. See [Requirements for OpenShift AI Self-Managed](#). For example:
 - If you plan to use NVIDIA GPUs, you must mirror deployed the NVIDIA GPU Operator. See [Configuring the NVIDIA GPU Operator](#) in the OpenShift Container Platform

[Installing the oc-mirror v2 plugin](#) in the OpenShift Container Platform documentation.

- If you plan to use the single-model serving platform to serve large models, you must mirror the Operators for Red Hat OpenShift Serverless and Red Hat OpenShift Service Mesh.
- If you plan to use the distributed workloads component, you must mirror the Ray cluster image.

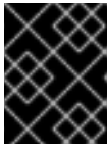


NOTE

This procedure uses the oc-mirror plugin v2; the oc-mirror plugin v1 is now deprecated. For more information, see [Changes from oc-mirror plugin v1 to v2](#) in the OpenShift documentation.

Procedure

1. Create a mirror registry. See [Creating a mirror registry with mirror registry for Red Hat OpenShift](#) in the OpenShift Container Platform documentation.
2. To mirror registry images, install the **oc-mirror** OpenShift CLI plugin v2 on your mirroring machine running Linux. See [Installing the oc-mirror OpenShift CLI plugin](#) in the OpenShift Container Platform documentation.



IMPORTANT

The oc-mirror plugin v1 is deprecated. Red Hat recommends that you use the oc-mirror plugin v2 for continued support and improvements.

3. Create a container image registry credentials file that allows mirroring images from Red Hat to your mirror. See [Configuring credentials that allow images to be mirrored](#) in the OpenShift Container Platform documentation.
4. Open the example image set configuration file (**rhoai-<version>.md**) from the [disconnected installer helper](#) repository and examine its contents.
The disconnected installer helper file includes a list of **Additional images** required to install OpenShift AI in a disconnected environment, as well as a list of older **Unsupported images** provided for reference only. These older images are no longer maintained by Red Hat but are included for convenience, such as when importing older resources or maintaining compatibility with previous environments.

5. Using the example image set configuration file, create a file called **imageset-config.yaml** and populate it with values suitable for the image set configuration in your deployment.

- To view a list of the available OpenShift versions, run the following command. This might take several minutes. If the command returns errors, repeat the steps in [Configuring credentials that allow images to be mirrored](#).

```
oc-mirror list operators
```

- To see the available channels for a package in a specific version of OpenShift Container Platform (for example, 4.18), run the following command:

```
oc-mirror list operators --catalog=registry.redhat.io/redhat/redhat-operator-index:v4.18 --package=<package_name>
```

- For information about subscription update channels, see [Understanding update channels](#).



IMPORTANT

The example image set configurations are for demonstration purposes only and might need further alterations depending on your deployment.

To identify the attributes most suitable for your deployment, see [Image set configuration parameters](#) and [Image set configuration examples](#) in the OpenShift Container Platform documentation.

The list of **Unsupported images** in the helper file is provided for reference only and should not be included in your mirrored image set unless you have a specific need to import older resources or maintain compatibility with previous environments.

Example imageset-config.yaml

```
kind: ImageSetConfiguration
apiVersion: mirror.openshift.io/v1alpha2
mirror:
  operators:
    - catalog: registry.redhat.io/redhat/redhat-operator-index:v4.19
      packages:
        - name: rhods-operator
          channels:
            - name: stable
              minVersion: 2.25.0
              maxVersion: 2.25.0
        - name: <additional_operator_name>
          channels:
            - name: stable
  additionalImages:
    - name: <additional_image_name>
```

6. Download the specified image set configuration to a local file on your mirroring machine:

- Replace **<mirror_rhoai>** with the target directory where you want to output the image set file.
- The target directory path must start with **file://**.
- The download might take several minutes.

```
$ oc mirror -c imageset-config.yaml file://<mirror_rhoai> --v2
```

TIP

If the **tls: failed to verify certificate: x509: certificate signed by unknown authority** error is returned and you want to ignore it, set **skipTLS** to **true** in your image set configuration file and run the command again.

7. Verify that the image set **.tar** files were created:

■

```
$ ls <mirror_rhoai>
```

Example output

```
mirror_000001.tar, mirror_000002.tar
```

If an **archiveSize** value was specified in the image set configuration file, the image set might be separated into multiple **.tar** files.

8. Optional: Verify that total size of the image set **.tar** files is around 75 GB:

```
$ du -h --max-depth=1 ./<mirror_rhoai>/
```

If the total size of the image set is significantly less than 75 GB, run the **oc mirror** command again.

9. Upload the contents of the generated image set to your target mirror registry:

- Replace **<mirror_rhoai>** with the directory that contains your image set **.tar** files.
- Replace **<registry.example.com:5000>** with your mirror registry.

```
$ oc mirror -c imageset-config.yaml --from file://<mirror_rhoai>
docker://<registry.example.com:5000> --v2
```

TIP

If the **tls: failed to verify certificate: x509: certificate signed by unknown authority** error is returned and you want to ignore it, run the following command:

```
$ oc mirror --dest-tls-verify false --from=./<mirror_rhoai>
docker://<registry.example.com:5000> --v2
```

10. Log in to your target OpenShift cluster using the OpenShift CLI as a user with the **cluster-admin** role.
11. Verify that the YAML files are present for the **ImageDigestMirrorSet** and **CatalogSource** resources:

- Replace **<mirror_rhoai>** with the directory that contains your image set **.tar** files.

```
$ ls <mirror_rhoai>/working-dir/cluster-resources/
```

Example output

```
cs-redhat-operator-index.yaml
idms-oc-mirror.yaml
```

12. Install the generated resources into the cluster:

- Replace **<oc_mirror_workspace_path>** with the path to your oc mirror workspace.

```
$ oc apply -f <oc_mirror_workspace_path>/working-dir/cluster-resources
```

Verification

- Verify that the **CatalogSource** and pod were created successfully:

```
$ oc get catalogsource,pod -n openshift-marketplace
```

This should return at least one catalog and two pods.

- Check that the Red Hat OpenShift AI Operator exists in the OperatorHub:
 - a. Log in to the OpenShift web console.
 - b. Click **Operators** → **OperatorHub**.
The **OperatorHub** page opens.
 - c. Confirm that the Red Hat OpenShift AI Operator is shown.
- If you mirrored additional operators, check that those operators exist in the OperatorHub.

Additional resources

[Mirroring images for a disconnected installation by using the oc-mirror plugin v2](#)

3.3. CONFIGURING CUSTOM NAMESPACES

By default, OpenShift AI uses the following predefined namespaces:

- **redhat-ods-operator** contains the Red Hat OpenShift AI Operator
- **redhat-ods-applications** includes the dashboard and other required components of OpenShift AI
- **rhods-notebooks** is where basic workbenches are deployed by default

If needed, you can define custom namespaces to use instead of the predefined ones before installing OpenShift AI. This flexibility supports environments with naming policies or conventions and allows cluster administrators to control where components such as workbenches are deployed.

Namespaces created by OpenShift AI typically include **openshift** or **redhat** in their name. Do not rename these system namespaces because they are required for OpenShift AI to function properly.

Prerequisites

- You have access to an OpenShift AI cluster with cluster administrator privileges.
- You have installed the OpenShift CLI (**oc**) as described in the appropriate documentation for your cluster:
 - [Installing the OpenShift CLI](#) for OpenShift Container Platform
 - [Installing the OpenShift CLI](#) for Red Hat OpenShift Service on AWS
- You have not yet installed the Red Hat OpenShift AI Operator.

Procedure

1. In a terminal window, if you are not already logged in to your OpenShift cluster as a cluster administrator, log in to the OpenShift CLI (**oc**) as shown in the following example:

```
oc login <openshift_cluster_url> -u <admin_username> -p <password>
```

2. Optional: To configure a custom operator namespace:

- a. Create a namespace YAML file named **operator-namespace.yaml**.

```
apiVersion: v1
kind: Namespace
metadata:
  name: <operator-namespace> 1
```

- 1 Defines the operator namespace.

- b. Create the namespace in your OpenShift cluster.

```
$ oc create -f operator-namespace.yaml
```

You see output similar to the following:

```
namespace/<operator-namespace> created
```

- c. When you install the Red Hat OpenShift AI Operator, use this namespace instead of **redhat-ods-operator**.

3. Optional: To configure a custom applications namespace:

- a. Create a namespace YAML file named **applications-namespace.yaml**.

```
apiVersion: v1
kind: Namespace
metadata:
  name: <applications-namespace> 1
  labels:
    opendatahub.io/application-namespace: 'true' 2
```

- 1 Defines the applications namespace.

- 2 Adds the required label.

- b. Create the namespace in your OpenShift cluster.

```
$ oc create -f applications-namespace.yaml
```

You see output similar to the following:

```
namespace/<applications-namespace> created
```

4. Optional: To configure a custom workbench namespace:

- a. Create a namespace YAML file named **workbench-namespace.yaml**.

```
apiVersion: v1
kind: Namespace
metadata:
  name: <workbench-namespace> 1
```

- 1** Defines the workbench namespace.

- b. Create the namespace in your OpenShift cluster.

```
$ oc create -f workbench-namespace.yaml
```

You see output similar to the following:

```
namespace/<workbench-namespace> created
```

- c. When you install the Red Hat OpenShift AI components, specify this namespace for the **spec.workbenches.workbenchNamespace** field. You cannot change the default workbench namespace after you have installed the Red Hat OpenShift AI Operator.

Next step

[Installing the Red Hat OpenShift AI Operator](#)

3.4. INSTALLING THE RED HAT OPENSIFT AI OPERATOR

This section shows how to install the Red Hat OpenShift AI Operator on your OpenShift cluster using the command-line interface (CLI) and the OpenShift web console.



NOTE

If you want to upgrade from a previous version of OpenShift AI rather than performing a new installation, see [Upgrading OpenShift AI in a disconnected environment](#).



NOTE

If your OpenShift cluster uses a proxy to access the Internet, you can configure the proxy settings for the Red Hat OpenShift AI Operator. See [Overriding proxy settings of an Operator](#) for more information.

3.4.1. Installing the Red Hat OpenShift AI Operator by using the CLI

The following procedure shows how to use the OpenShift CLI (**oc**) to install the Red Hat OpenShift AI Operator on your OpenShift cluster. You must install the Operator before you can install OpenShift AI components on the cluster.

Prerequisites

- You have a running OpenShift cluster, version 4.16 or greater, configured with a default storage class that can be dynamically provisioned.

- You have cluster administrator privileges for your OpenShift cluster.
- You have installed the OpenShift CLI (**oc**) as described in the appropriate documentation for your cluster:
 - [Installing the OpenShift CLI](#) for OpenShift Container Platform
 - [Installing the OpenShift CLI](#) for Red Hat OpenShift Service on AWS
- If you are using custom namespaces, you have created and labeled them as required.



NOTE

The example commands in this procedure use the predefined operator namespace. If you are using a custom operator namespace, replace **redhat-ods-operator** with your namespace.

- You have mirrored the required container images to a private registry. See [Mirroring images to a private registry for a disconnected installation](#).

Procedure

1. Open a new terminal window.
2. Follow these steps to log in to your OpenShift cluster as a cluster administrator:
 - a. In the upper-right corner of the OpenShift web console, click your user name and select **Copy login command**.
 - b. After you have logged in, click **Display token**.
 - c. Copy the **Log in with this token** command and paste it in your terminal.

```
$ oc login --token=<token> --server=<openshift_cluster_url>
```

3. Create a namespace for installation of the Operator by performing the following actions:



NOTE

If you have already created a custom namespace for the Operator, you can skip this step.

- a. Create a namespace YAML file named **rhods-operator-namespace.yaml**.

```
apiVersion: v1
kind: Namespace
metadata:
  name: redhat-ods-operator 1
```

- 1 Defines the operator namespace.

- b. Create the namespace in your OpenShift cluster.

```
$ oc create -f rhods-operator-namespace.yaml
```

You see output similar to the following:

```
namespace/redhat-ods-operator created
```

4. Create an operator group for installation of the Operator by performing the following actions:

- a. Create an **OperatorGroup** object custom resource (CR) file, for example, **rhods-operator-group.yaml**.

```
apiVersion: operators.coreos.com/v1
kind: OperatorGroup
metadata:
  name: rhods-operator
  namespace: redhat-ods-operator 1
```

- 1 Defines the operator namespace.

- b. Create the **OperatorGroup** object in your OpenShift cluster.

```
$ oc create -f rhods-operator-group.yaml
```

You see output similar to the following:

```
operatorgroup.operators.coreos.com/rhods-operator created
```

5. Create a subscription for installation of the Operator by performing the following actions:

- a. Create a **Subscription** object CR file, for example, **rhods-operator-subscription.yaml**.

```
apiVersion: operators.coreos.com/v1alpha1
kind: Subscription
metadata:
  name: rhods-operator
  namespace: redhat-ods-operator 1
spec:
  name: rhods-operator
  channel: <channel> 2
  source: cs-redhat-operator-index
  sourceNamespace: openshift-marketplace
  startingCSV: rhods-operator.x.y.z 3
```

- 1 Defines the operator namespace.

- 2 Sets the update channel. You must specify a value of **fast**, **stable**, **stable-x.y**, **eus-x.y**, or **alpha**. For more information, see [Understanding update channels](#).

- 3 Optional: Sets the operator version. If you do not specify a value, the subscription defaults to the latest operator version. For more information, see the [Red Hat OpenShift AI Self-Managed Life Cycle](#) Knowledgebase article.

- b. Create the **Subscription** object in your OpenShift cluster to install the Operator.

```
$ oc create -f rhods-operator-subscription.yaml
```

You see output similar to the following:

```
subscription.operators.coreos.com/rhods-operator created
```

Verification

- In the OpenShift web console, click **Operators → Installed Operators** and confirm that the Red Hat OpenShift AI Operator shows one of the following statuses:
 - **Installing** – installation is in progress; wait for this to change to **Succeeded**. This might take several minutes.
 - **Succeeded** – installation is successful.

Next step

- [Installing and managing Red Hat OpenShift AI components](#)

Additional resources

- [Adding users to OpenShift AI user groups](#)
- [Adding Operators to a cluster](#)

3.4.2. Installing the Red Hat OpenShift AI Operator by using the web console

The following procedure shows how to use the OpenShift web console to install the Red Hat OpenShift AI Operator on your cluster. You must install the Operator before you can install OpenShift AI components on the cluster.

Prerequisites

- You have a running OpenShift cluster, version 4.16 or greater, configured with a default storage class that can be dynamically provisioned.
- You have cluster administrator privileges for your OpenShift cluster.
- If you are using custom namespaces, you have created and labeled them as required.
- You have mirrored the required container images to a private registry. See [Mirroring images to a private registry for a disconnected installation](#).

Procedure

1. Log in to the OpenShift web console as a cluster administrator.
2. In the web console, click **Operators → OperatorHub**.
3. On the **OperatorHub** page, locate the Red Hat OpenShift AI Operator by scrolling through the available Operators or by typing *Red Hat OpenShift AI* into the **Filter by keyword** box.

4. Click the **Red Hat OpenShift AI** tile. The **Red Hat OpenShift AI** information pane opens.
5. Select a **Channel**. For information about subscription update channels, see [Understanding update channels](#).
6. Select a **Version**.
7. Click **Install**. The **Install Operator** page opens.
8. Review or change the selected channel and version as needed.
9. For **Installation mode**, note that the only available value is **All namespaces on the cluster (default)**. This installation mode makes the Operator available to all namespaces in the cluster.
10. For **Installed Namespace**, choose one of the following options:
 - To use the predefined operator namespace, select the **Operator recommended Namespace: redhat-ods-operator** option.
 - To use the custom operator namespace that you created, select the **Select a Namespace** option, and then select the namespace from the drop-down list.
11. For **Update approval**, select one of the following update strategies:
 - **Automatic**: Your environment attempts to install new updates when they are available based on the content of your mirror.
 - **Manual**: A cluster administrator must approve any new updates before installation begins.



IMPORTANT

By default, the Red Hat OpenShift AI Operator follows a sequential update process. This means that if there are several versions between the current version and the target version, Operator Lifecycle Manager (OLM) upgrades the Operator to each of the intermediate versions before it upgrades it to the final, target version.

If you configure automatic upgrades, OLM automatically upgrades the Operator to the *latest* available version. If you configure manual upgrades, a cluster administrator must manually approve each sequential update between the current version and the final, target version.

For information about supported versions, see the [Red Hat OpenShift AI Life Cycle](#) Knowledgebase article.

12. Click **Install**.
The **Installing Operators** pane appears. When the installation finishes, a checkmark appears next to the Operator name.

Verification

- In the OpenShift web console, click **Operators → Installed Operators** and confirm that the Red Hat OpenShift AI Operator shows one of the following statuses:
 - **Installing** – installation is in progress; wait for this to change to **Succeeded**. This might take several minutes.

- **Succeeded** – installation is successful.

Next step

- [Installing and managing Red Hat OpenShift AI components](#)

Additional resources

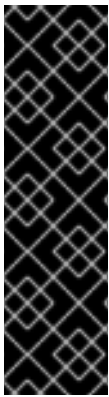
- [Adding users to OpenShift AI user groups](#)
- [Adding Operators to a cluster](#)

3.5. INSTALLING AND MANAGING RED HAT OPENSIFT AI COMPONENTS

You can use the OpenShift command-line interface (CLI) or OpenShift web console to install and manage components of Red Hat OpenShift AI on your OpenShift cluster.

3.5.1. Installing Red Hat OpenShift AI components by using the CLI

To install Red Hat OpenShift AI components by using the OpenShift CLI (**oc**), you must create and configure a **DataScienceCluster** object.



IMPORTANT

The following procedure describes how to create and configure a **DataScienceCluster** object to install Red Hat OpenShift AI components as part of a *new* installation.

- For information about changing the installation status of OpenShift AI components *after* installation, see [Updating the installation status of Red Hat OpenShift AI components by using the web console](#).
- For information about *upgrading* OpenShift AI, see [Upgrading OpenShift AI Self-Managed in a disconnected environment](#).

Prerequisites

- The Red Hat OpenShift AI Operator is installed on your OpenShift cluster. See [Installing the Red Hat OpenShift AI Operator](#).
- You have cluster administrator privileges for your OpenShift cluster.
- You have installed the OpenShift CLI (**oc**) as described in the appropriate documentation for your cluster:
 - [Installing the OpenShift CLI](#) for OpenShift Container Platform
 - [Installing the OpenShift CLI](#) for Red Hat OpenShift Service on AWS
- If you are using custom namespaces, you have created the namespaces.

Procedure

1. Open a new terminal window.

2. Follow these steps to log in to your OpenShift cluster as a cluster administrator:
 - a. In the upper-right corner of the OpenShift web console, click your user name and select **Copy login command**.
 - b. After you have logged in, click **Display token**.
 - c. Copy the **Log in with this token** command and paste it in your terminal.

```
$ oc login --token=<token> --server=<openshift_cluster_url>
```

3. Create a **DataScienceCluster** object custom resource (CR) file, for example, **rhods-operator-dsc.yaml**.

```
apiVersion: datasciencecluster.opendatahub.io/v1
kind: DataScienceCluster
metadata:
  name: default-dsc
spec:
  components:
    codeflare:
      managementState: Removed
    dashboard:
      managementState: Removed
    datasciencepipelines:
      argoWorkflowsControllers:
        managementState: Removed 1
        managementState: Removed
    feastoperator:
      managementState: Removed
    kserve:
      managementState: Removed 2 3
    kueue:
      defaultClusterQueueName: default
      defaultLocalQueueName: default
      managementState: Removed
    llamastackoperator:
      managementState: Removed
    modelmeshserving:
      managementState: Removed
    modelregistry:
      managementState: Removed
      registriesNamespace: {mr-default-namespace}
    ray:
      managementState: Removed
    trainingoperator:
      managementState: Removed
    trustyai:
      managementState: Removed
    workbenches:
      managementState: Removed
      workbenchNamespace: {workbench-default-namespace} 4
```

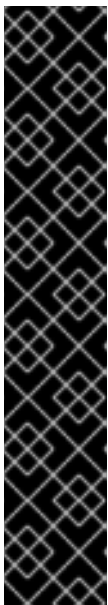
- 1 To use your own Argo Workflows instance with the **datasciencepipelines** component, set **argoWorkflowsControllers.managementState** to **Removed**. This allows you to integrate with a managed Argo Workflows installation already on your OpenShift cluster and avoid conflicts with the embedded controller. See *Configuring pipelines with your own Argo Workflows instance*.
 - 2 To fully install the KServe component, which is used by the single-model serving platform to serve large models, you must install Operators for Red Hat OpenShift Service Mesh and Red Hat OpenShift Serverless and perform additional configuration. See [Installing the single-model serving platform](#).
 - 3 If you have *not* enabled the KServe component (that is, you set the value of the **managementState** field to **Removed**), you must also disable the dependent Service Mesh component to avoid errors. See [Disabling KServe dependencies](#).
 - 4 To use the predefined workbench namespace, set this value to **rhods-notebooks** or omit this line. To use a custom workbench namespace, set this value to your namespace.
4. In the **spec.components** section of the CR, for each OpenShift AI component shown, set the value of the **managementState** field to either **Managed** or **Removed**. These values are defined as follows:

Managed

The Operator actively manages the component, installs it, and tries to keep it active. The Operator will upgrade the component only if it is safe to do so.

Removed

The Operator actively manages the component but does not install it. If the component is already installed, the Operator will try to remove it.



IMPORTANT

- To learn how to fully install the KServe component, which is used by the single-model serving platform to serve large models, see [Installing the single-model serving platform](#).
- If you have *not* enabled the KServe component (that is, you set the value of the **managementState** field to **Removed**), you must also disable the dependent Service Mesh component to avoid errors. See [Disabling KServe dependencies](#).
- To learn how to install the distributed workloads components, see [Installing the distributed workloads components](#).
- To learn how to run distributed workloads in a disconnected environment, see [Running distributed data science workloads in a disconnected environment](#).

5. Create the **DataScienceCluster** object in your OpenShift cluster to install the specified OpenShift AI components.

```
$ oc create -f rhods-operator-dsc.yaml
```

You see output similar to the following:

datasciencecluster.datasciencecluster.opendatahub.io/default created

Verification

1. Confirm that there is at least one running pod for each component:
 - a. In the OpenShift web console, click **Workloads → Pods**.
 - b. In the **Project** list at the top of the page, select **redhat-ods-applications**.
 - c. In the applications namespace, confirm that there are one or more running pods for each of the OpenShift AI components that you installed.
2. Confirm the status of all installed components:
 - a. In the OpenShift web console, click **Operators → Installed Operators**.
 - b. Click the **Red Hat OpenShift AI Operator**.
 - c. Click the **Data Science Cluster** tab.
 - d. For the **DataScienceCluster** object called **default-dsc**, verify that the status is **Phase: Ready**.



NOTE

When you edit the **spec.components** section to change the installation status of a component, the **default-dsc** status also changes. During the initial installation, it might take a few minutes for the status phase to change from **Progressing** to **Ready**. You can access the OpenShift AI dashboard before the **default-dsc** status phase is **Ready**, but all components might not be ready.

- e. Click the **default-dsc** link to display the data science cluster details.
- f. Select the **YAML** tab.
- g. In the **status.installedComponents** section, confirm that the components you installed have a status value of **true**.



NOTE

If a component shows with the **component-name: {}** format in the **spec.components** section of the CR, the component is not installed.

3. In the OpenShift AI dashboard, users can view the list of the installed OpenShift AI components, their corresponding source (upstream) components, and the versions of the installed components, as described in [Viewing installed OpenShift AI components](#).

3.5.2. Installing Red Hat OpenShift AI components by using the web console

To install Red Hat OpenShift AI components by using the OpenShift web console, you must create and configure a **DataScienceCluster** object.



IMPORTANT

The following procedure describes how to create and configure a **DataScienceCluster** object to install Red Hat OpenShift AI components as part of a *new* installation.

- For information about changing the installation status of OpenShift AI components *after* installation, see [Updating the installation status of Red Hat OpenShift AI components by using the web console](#).
- For information about *upgrading* OpenShift AI, see [Upgrading OpenShift AI Self-Managed in a disconnected environment](#).

Prerequisites

- The Red Hat OpenShift AI Operator is installed on your OpenShift cluster. See [Installing the Red Hat OpenShift AI Operator](#).
- You have cluster administrator privileges for your OpenShift cluster.
- If you are using custom namespaces, you have created the namespaces.

Procedure

1. Log in to the OpenShift web console as a cluster administrator.
2. In the web console, click **Operators** → **Installed Operators** and then click the **Red Hat OpenShift AI** Operator.
3. Click the **Data Science Cluster** tab.
4. Click **Create DataScienceCluster**.
5. For **Configure via**, select **YAML view**.

An embedded YAML editor opens showing a default custom resource (CR) for the **DataScienceCluster** object, similar to the following example:

```
apiVersion: datasciencecluster.opendatahub.io/v1
kind: DataScienceCluster
metadata:
  name: default-dsc
spec:
  components:
    codeflare:
      managementState: Removed
    dashboard:
      managementState: Removed
    datasciencepipelines:
      argoWorkflowsControllers:
        managementState: Removed 1
        managementState: Removed
    feastoperator:
      managementState: Removed
    kserve:
      managementState: Removed 2 3
    kueue:
      defaultClusterQueueName: default
```

```

defaultLocalQueueName: default
managementState: Removed
llamastackoperator:
  managementState: Removed
modelmeshserving:
  managementState: Removed
modelregistry:
  managementState: Removed
  registriesNamespace: {mr-default-namespace}
ray:
  managementState: Removed
trainingoperator:
  managementState: Removed
trustyai:
  managementState: Removed
workbenches:
  managementState: Removed
  workbenchNamespace: {workbench-default-namespace} 4

```

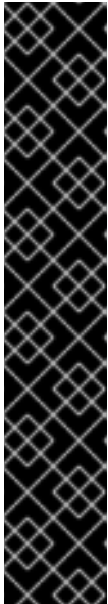
- 1 To use your own Argo Workflows instance with the **datasciencepipelines** component, set **argoWorkflowsControllers.managementState** to **Removed**. This allows you to integrate with a managed Argo Workflows installation already on your OpenShift cluster and avoid conflicts with the embedded controller. See *Configuring pipelines with your own Argo Workflows instance*.
 - 2 To fully install the KServe component, which is used by the single-model serving platform to serve large models, you must install Operators for Red Hat OpenShift Service Mesh and Red Hat OpenShift Serverless and perform additional configuration. See [Installing the single-model serving platform](#).
 - 3 If you have *not* enabled the KServe component (that is, you set the value of the **managementState** field to **Removed**), you must also disable the dependent Service Mesh component to avoid errors. See [Disabling KServe dependencies](#).
 - 4 To use the predefined workbench namespace, set this value to **rhods-notebooks** or omit this line. To use a custom workbench namespace, set this value to your namespace.
6. In the **spec.components** section of the CR, for each OpenShift AI component shown, set the value of the **managementState** field to either **Managed** or **Removed**. These values are defined as follows:

Managed

The Operator actively manages the component, installs it, and tries to keep it active. The Operator will upgrade the component only if it is safe to do so.

Removed

The Operator actively manages the component but does not install it. If the component is already installed, the Operator will try to remove it.



IMPORTANT

- To learn how to fully install the KServe component, which is used by the single-model serving platform to serve large models, see [Installing the single-model serving platform](#).
- If you have *not* enabled the KServe component (that is, you set the value of the **managementState** field to **Removed**), you must also disable the dependent Service Mesh component to avoid errors. See [Disabling KServe dependencies](#).
- To learn how to install the distributed workloads components, see [Installing the distributed workloads components](#).
- To learn how to run distributed workloads in a disconnected environment, see [Running distributed data science workloads in a disconnected environment](#).

7. Click **Create**.

Verification

1. Confirm the status of all installed components:
 - a. In the OpenShift web console, click **Operators → Installed Operators**.
 - b. Click the **Red Hat OpenShift AI Operator**.
 - c. Click the **Data Science Cluster** tab.
 - d. For the **DataScienceCluster** object called **default-dsc**, verify that the status is **Phase: Ready**.



NOTE

When you edit the **spec.components** section to change the installation status of a component, the **default-dsc** status also changes. During the initial installation, it might take a few minutes for the status phase to change from **Progressing** to **Ready**. You can access the OpenShift AI dashboard before the **default-dsc** status phase is **Ready**, but all components might not be ready.

- e. Click the **default-dsc** link to display the data science cluster details.
- f. Select the **YAML** tab.
- g. In the **status.installedComponents** section, confirm that the components you installed have a status value of **true**.



NOTE

If a component shows with the **component-name: {}** format in the **spec.components** section of the CR, the component is not installed.

2. Confirm that there is at least one running pod for each component:
 - a. In the OpenShift web console, click **Workloads** → **Pods**.
 - b. In the **Project** list at the top of the page, select **redhat-ods-applications** or your custom applications namespace.
 - c. In the applications namespace, confirm that there are one or more running pods for each of the OpenShift AI components that you installed.
3. In the OpenShift AI dashboard, users can view the list of the installed OpenShift AI components, their corresponding source (upstream) components, and the versions of the installed components, as described in [Viewing installed OpenShift AI components](#).

3.5.3. Updating the installation status of Red Hat OpenShift AI components by using the web console

You can use the OpenShift web console to update the installation status of components of Red Hat OpenShift AI on your OpenShift cluster.



IMPORTANT

If you upgraded OpenShift AI, the upgrade process automatically used the values of the previous version's **DataScienceCluster** object. New components are not automatically added to the **DataScienceCluster** object.

After upgrading OpenShift AI:

- Inspect the default **DataScienceCluster** object to check and optionally update the **managementState** status of the existing components.
- Add any new components to the **DataScienceCluster** object.

Prerequisites

- The Red Hat OpenShift AI Operator is [installed](#) on your OpenShift cluster.
- You have cluster administrator privileges for your OpenShift cluster.

Procedure

1. Log in to the OpenShift web console as a cluster administrator.
2. In the web console, click **Operators** → **Installed Operators** and then click the **Red Hat OpenShift AI** Operator.
3. Click the **Data Science Cluster** tab.
4. On the **DataScienceClusters** page, click the **default-dsc** object.
5. Click the **YAML** tab.
An embedded YAML editor opens showing the default custom resource (CR) for the **DataScienceCluster** object, similar to the following example:

```
apiVersion: datasciencecluster.opendatahub.io/v1
```

```

kind: DataScienceCluster
metadata:
  name: default-dsc
spec:
  components:
    codeflare:
      managementState: Removed
    dashboard:
      managementState: Removed
    datasciencepipelines:
      managementState: Removed
    kserve:
      managementState: Removed
    kueue:
      managementState: Removed
    llamastackoperator:
      managementState: Removed
    modelmeshserving:
      managementState: Removed
    ray:
      managementState: Removed
    trainingoperator:
      managementState: Removed
    trustyai:
      managementState: Removed
    workbenches:
      managementState: Removed
      workbenchNamespace: rhods-notebooks

```

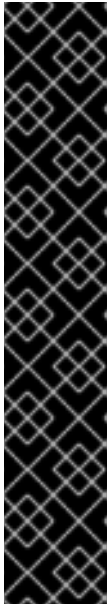
6. In the **spec.components** section of the CR, for each OpenShift AI component shown, set the value of the **managementState** field to either **Managed** or **Removed**. These values are defined as follows:

Managed

The Operator actively manages the component, installs it, and tries to keep it active. The Operator will upgrade the component only if it is safe to do so.

Removed

The Operator actively manages the component but does not install it. If the component is already installed, the Operator will try to remove it.



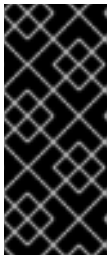
IMPORTANT

- To learn how to install the KServe component, which is used by the single-model serving platform to serve large models, see [Installing the single-model serving platform](#).
- If you have *not* enabled the KServe component (that is, you set the value of the **managementState** field to **Removed**), you must also disable the dependent Service Mesh component to avoid errors. See [Disabling KServe dependencies](#).
- To learn how to install the distributed workloads feature, see [Installing the distributed workloads components](#).
- To learn how to run distributed workloads in a disconnected environment, see [Running distributed data science workloads in a disconnected environment](#).

7. Click **Save**.

For any components that you updated, OpenShift AI initiates a rollout that affects all pods to use the updated image.

8. If you are upgrading from OpenShift AI 2.19 or earlier, upgrade the Authorino Operator to the **stable** update channel, version 1.2.1 or later.



IMPORTANT

If you are upgrading the Authorino Operator to the **stable** update channel, version 1.2.1 or later in a disconnected environment, use the following upgrade procedure described in the release notes: [RHOAIENG-24786 - Upgrading the Authorino Operator from Technical Preview to Stable fails in disconnected environments](#). Otherwise, the upgrade can fail.

Verification

1. Confirm that there is at least one running pod for each component:
 - a. In the OpenShift web console, click **Workloads → Pods**.
 - b. In the **Project** list at the top of the page, select **redhat-ods-applications** or your custom applications namespace.
 - c. In the applications namespace, confirm that there are one or more running pods for each of the OpenShift AI components that you installed.
2. Confirm the status of all installed components:
 - a. In the OpenShift web console, click **Operators → Installed Operators**.
 - b. Click the **Red Hat OpenShift AI Operator**.
 - c. Click the **Data Science Cluster** tab and select the **DataScienceCluster** object called **default-dsc**.
 - d. Select the **YAML** tab.

- e. In the **status.installedComponents** section, confirm that the components you installed have a status value of **true**.



NOTE

If a component shows with the **component-name: {}** format in the **spec.components** section of the CR, the component is not installed.

3. In the OpenShift AI dashboard, users can view the list of the installed OpenShift AI components, their corresponding source (upstream) components, and the versions of the installed components, as described in [Viewing installed OpenShift AI components](#).

3.5.4. Viewing installed OpenShift AI components

In the Red Hat OpenShift AI dashboard, you can view a list of the installed OpenShift AI components, their corresponding source (upstream) components, and the versions of the installed components.

Prerequisites

- OpenShift AI is installed in your OpenShift cluster.

Procedure

1. Log in to the OpenShift AI dashboard.
2. In the top navigation bar, click the help icon () and then select **About**.

Verification

The **About** page shows a list of the installed OpenShift AI components along with their corresponding upstream components and upstream component versions.

Additional resources

- [Installing and managing Red Hat OpenShift AI components](#).

CHAPTER 4. CONFIGURING PIPELINES WITH YOUR OWN ARGO WORKFLOWS INSTANCE

You can configure OpenShift AI to use an existing Argo Workflows instance instead of the embedded one included with Data Science Pipelines. This configuration is useful if your OpenShift cluster already includes a managed Argo Workflows instance and you want to integrate it with OpenShift AI pipelines without conflicts. Disabling the embedded Argo Workflows controller allows cluster administrators to manage the lifecycles of OpenShift AI and Argo Workflows independently.



NOTE

You cannot enable both the embedded Argo Workflows instance and your own Argo Workflows instance on the same cluster.

Prerequisites

- You have cluster administrator privileges for your OpenShift cluster.
- You have installed Red Hat OpenShift AI.

Procedure

1. Log in to the OpenShift web console as a cluster administrator.
2. In the OpenShift console, click **Operators → Installed Operators**.
3. Search for the **Red Hat OpenShift AI** Operator, and then click the Operator name to open the Operator details page.
4. Click the **Data Science Cluster** tab.
5. Click the default instance name (for example, **default-dsc**) to open the instance details page.
6. Click the **YAML** tab to show the instance specifications.
7. Disable the embedded Argo Workflows controllers that are managed by the OpenShift AI Operator:
 - a. In the **spec.components** section, set the value of the **managementState** field for the **datasciencepipelines** component to **Managed**.
 - b. In the **spec.components.datasciencepipelines** section, set the value of the **managementState** field for **argoWorkflowsControllers** to **Removed**, as shown in the following example:

Example datasciencepipelines specification

```
# ...
spec:
  components:
    datasciencepipelines:
      argoWorkflowsControllers:
        managementState: Removed
        managementState: Managed
# ...
```

-
- 8. Click **Save** to apply your changes.
- 9. Install and configure a compatible version of Argo Workflows on your cluster. For compatible version information, see [Supported Configurations](#). For installation information, see the [Argo Workflows Installation documentation](#).

Verification

1. On the **Details** tab of the **DataScienceCluster** instance (for example, **default-dsc**), verify that **DataSciencePipelinesReady** has a **Status** of **True**.
2. Verify that the **ds-pipeline-workflow-controller** pod does not exist:
 - a. Go to **Workloads** → **Pods**.
 - b. Search for the **ds-pipeline-workflow-controller** pod.
 - c. Verify that this pod does not exist. The absence of this pod confirms that the embedded Argo Workflows controller is disabled.

CHAPTER 5. INSTALLING THE DISTRIBUTED WORKLOADS COMPONENTS

To use the distributed workloads feature in OpenShift AI, you must install several components.

Prerequisites

- You have logged in to OpenShift with the **cluster-admin** role and you can access the data science cluster.
- You have installed Red Hat OpenShift AI.
- You have installed the Red Hat build of Kueue Operator on your OpenShift cluster, as described in the [Red Hat build of Kueue documentation](#).
- You have sufficient resources. In addition to the minimum OpenShift AI resources described in [Installing and deploying OpenShift AI](#) (for disconnected environments, see [Deploying OpenShift AI in a disconnected environment](#)), you need 1.6 vCPU and 2 GiB memory to deploy the distributed workloads infrastructure.
- You have removed any previously installed instances of the CodeFlare Operator, as described in the Knowledgebase solution [How to migrate from a separately installed CodeFlare Operator in your data science cluster](#).
- If you want to use graphics processing units (GPUs), you have enabled GPU support in OpenShift AI. If you use NVIDIA GPUs, see [Enabling NVIDIA GPUs](#). If you use AMD GPUs, see [AMD GPU integration](#).



NOTE

In OpenShift AI, Red Hat supports the use of accelerators within the same cluster only.

Starting from Red Hat OpenShift AI 2.19, Red Hat supports remote direct memory access (RDMA) for NVIDIA GPUs only, enabling them to communicate directly with each other by using NVIDIA GPUDirect RDMA across either Ethernet or InfiniBand networks.

- If you want to use self-signed certificates, you have added them to a central Certificate Authority (CA) bundle as described in [Working with certificates](#) (for disconnected environments, see [Working with certificates](#)). No additional configuration is necessary to use those certificates with distributed workloads. The centrally configured self-signed certificates are automatically available in the workload pods at the following mount points:

- Cluster-wide CA bundle:

```
/etc/pki/tls/certs/odh-trusted-ca-bundle.crt
/etc/ssl/certs/odh-trusted-ca-bundle.crt
```

- Custom CA bundle:

```
/etc/pki/tls/certs/odh-ca-bundle.crt
/etc/ssl/certs/odh-ca-bundle.crt
```


Procedure

1. In the OpenShift console, click **Operators** → **Installed Operators**.
2. Search for the **Red Hat OpenShift AI** Operator, and then click the Operator name to open the Operator details page.
3. Click the **Data Science Cluster** tab.
4. Click the default instance name (for example, **default-dsc**) to open the instance details page.
5. Click the **YAML** tab to show the instance specifications.
6. Enable the required distributed workloads components. In the **spec.components** section, set the **managementState** field correctly for the required components:
 - Set **kueue** to **Unmanaged** to allow the Red Hat build of Kueue Operator to manage Kueue.
 - If you want to use the CodeFlare framework to tune models, set **codeflare** and **ray** to **Managed**.
 - If you want to use the Kubeflow Training Operator to tune models, set **trainingoperator** to **Managed**.
 - The list of required components depends on whether the distributed workload is run from a pipeline or workbench or both, as shown in the following table.

Table 5.1. Components required for distributed workloads

Component	Pipelines only	Workbenches only	Pipelines and workbenches
codeflare	Managed	Managed	Managed
dashboard	Managed	Managed	Managed
datasciencepipelines	Managed	Removed	Managed
kueue	Unmanaged	Unmanaged	Unmanaged
ray	Managed	Managed	Managed
trainingoperator	Managed	Managed	Managed
workbenches	Removed	Managed	Managed

7. Click **Save**. After a short time, the components with a **Managed** state are ready.

Verification

Check the status of the **codeflare-operator-manager**, **kubeflow-training-operator**, **kuberay-operator**, **kueue-controller-manager**, and **openshift-kueue-operator** pods, as follows:

1. In the OpenShift console, click **Workloads** → **Deployments**.
2. In the **Search by name** field, enter the following search strings:
 - In the **redhat-ods-applications** project, search for **codeflare-operator-manager**, **kubeflow-training-operator**, and **kuberay-operator**.
 - In the **openshift-kueue-operator** project, search for **kueue-controller-manager** and **openshift-kueue-operator**.
3. In each case, check the status as follows:
 - a. Click the deployment name to open the deployment details page.
 - b. Click the **Pods** tab.
 - c. Check the pod status.
When the status of the pods is **Running**, the pods are ready to use.
 - d. To see more information about each pod, click the pod name to open the pod details page, and then click the **Logs** tab.

Next Step

Configure the distributed workloads feature as described in [Managing distributed workloads](#).

CHAPTER 6. INSTALLING THE SINGLE-MODEL SERVING PLATFORM

6.1. ABOUT THE SINGLE-MODEL SERVING PLATFORM

For deploying large models such as large language models (LLMs), OpenShift AI includes a single-model serving platform that is based on the [KServe](#) component. To install the single-model serving platform, the following components are required:

- [KServe](#): A Kubernetes custom resource definition (CRD) that orchestrates model serving for all types of models. KServe includes model-serving runtimes that implement the loading of given types of model servers. KServe also handles the lifecycle of the deployment object, storage access, and networking setup.
- [Red Hat OpenShift Serverless](#): A cloud-native development model that allows for serverless deployments of models. OpenShift Serverless is based on the open source [Knative](#) project.
- [Red Hat OpenShift Service Mesh](#): A service mesh networking layer that manages traffic flows and enforces access policies. OpenShift Service Mesh is based on the open source [Istio](#) project.



NOTE

Currently, only OpenShift Service Mesh v2 is supported. For more information, see [Supported Configurations](#).

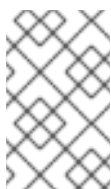
You can install the single-model serving platform manually or in an automated fashion:

Automated installation

If you have not already created a **ServiceMeshControlPlane** or **KNativeServing** resource on your OpenShift cluster, you can configure the Red Hat OpenShift AI Operator to install KServe and configure its dependencies. For more information, see [Configuring automated installation of KServe](#).

Manual installation

If you have already created a **ServiceMeshControlPlane** or **KNativeServing** resource on your OpenShift cluster, you *cannot* configure the Red Hat OpenShift AI Operator to install KServe and configure its dependencies. In this situation, you must install KServe manually. For more information, see [Manually installing KServe](#).



NOTE

You can run KServe in **Unmanaged** mode during manual installations of the single-model serving platform. This mode is useful when you need more control over KServe components, such as modifying resource limits for the KServe controller.

6.2. CONFIGURING AUTOMATED INSTALLATION OF KSERVE

If you have not already created a **ServiceMeshControlPlane** or **KNativeServing** resource on your OpenShift cluster, you can configure the Red Hat OpenShift AI Operator to install KServe and configure its dependencies.

You can configure KServe in **advanced** or **standard** deployment mode. For more information, see

[About KServe deployment modes](#). If you configure KServe for **advanced** deployment mode, you can set up your data science project to serve models in both **advanced** and **standard** deployment mode. However, if you configure KServe for only **standard** deployment mode, you can only use **standard** deployment mode.



IMPORTANT

If you have created a **ServiceMeshControlPlane** or **KNativeServing** resource on your cluster, the Red Hat OpenShift AI Operator cannot install KServe and configure its dependencies and the installation does not proceed. In this situation, you must follow the manual installation instructions to install KServe.

Prerequisites

- You have cluster administrator privileges for your OpenShift cluster.
- Your cluster has a node with 4 CPUs and 16 GB memory.
- You have installed the OpenShift CLI (**oc**) as described in the appropriate documentation for your cluster:
 - [Installing the OpenShift CLI](#) for OpenShift Container Platform
 - [Installing the OpenShift CLI](#) for Red Hat OpenShift Service on AWS
- (Knative Serverless mode): You have [installed](#) the Red Hat OpenShift Service Mesh Operator and dependent Operators.



NOTE

To enable automated installation of KServe, install *only* the required Operators for Red Hat OpenShift Service Mesh. Do not perform any additional configuration or create a **ServiceMeshControlPlane** resource.

- (Knative Serverless mode): You have [installed](#) the Red Hat OpenShift Serverless Operator.



NOTE

To enable automated installation of KServe, install *only* the Red Hat OpenShift Serverless Operator. Do not perform any additional configuration or create a **KNativeServing** resource.

- You have [installed](#) the Red Hat OpenShift AI Operator and [created](#) a **DataScienceCluster** object.
- (Knative Serverless mode): To add Authorino as an authorization provider so that you can enable token authentication for deployed models, you have installed the **Red Hat - Authorino** Operator. See [Installing the Authorino Operator](#).

Procedure

1. Log in to the OpenShift web console as a cluster administrator.

2. In the web console, click **Operators → Installed Operators** and then click the Red Hat OpenShift AI Operator.
3. Install OpenShift Service Mesh as follows:
 - a. Click the **DSC Initialization** tab.
 - b. Click the **default-dsci** object.
 - c. Click the **YAML** tab.
 - d. (Knative Serverless mode): In the **spec** section, validate that the value of the **managementState** field for the **serviceMesh** component is set to **Managed**, as shown:

```
spec:
  applicationsNamespace: redhat-ods-applications
  serviceMesh:
    controlPlane:
      metricsCollection: Istio
      name: data-science-smcp
      namespace: istio-system
      managementState: Managed
```

- e. (KServe RawDeployment mode): In the **spec** section, validate that the value of the **managementState** field for the **serviceMesh** component is set to **Removed**, as shown:

```
spec:
  applicationsNamespace: redhat-ods-applications
  serviceMesh:
    controlPlane:
      metricsCollection: Istio
      name: data-science-smcp
      namespace: istio-system
      managementState: Removed
```



NOTE

Do not change the **istio-system** namespace that is specified for the **serviceMesh** component by default. Other namespace values are not supported.

- f. Click **Save**.
Based on the configuration you added to the **DSCInitialization** object, the Red Hat OpenShift AI Operator installs OpenShift Service Mesh.
4. (KServe RawDeployment mode only): Install KServe as follows:
 - a. In the web console, click **Operators → Installed Operators** and then click the Red Hat OpenShift AI Operator.
 - b. Click the **Data Science Cluster** tab.
 - c. Click the **default-dsc** DSC object.
 - d. Click the **YAML** tab.

- e. In the **spec.components** section, configure the **kserve** component as shown.

```
kserve:
  defaultDeploymentMode: RawDeployment
  rawDeploymentServiceConfig: Headed 1
  managementState: Managed
  serving:
    managementState: Removed 2
    name: knative-serving
```

- f. Click **Save**.

The preceding configuration installs KServe in the KServe RawDeployment mode. In this configuration, observe the following details:

- 1 The configuration shown uses the **Headed** mode to set the cluster to perform normal load balancing over workload replicas. For environments in which inference request load balancing is performed on the client side, set **rawDeploymentServiceConfig** to the **Headless** mode.
- 2 The **managementState** is set to **Removed**.

5. (Knative Serverless mode): Install both KServe and OpenShift Serverless as follows:

- a. In the web console, click **Operators → Installed Operators** and then click the Red Hat OpenShift AI Operator.
- b. Click the **Data Science Cluster** tab.
- c. Click the **default-dsc** DSC object.
- d. Click the **YAML** tab.
- e. In the **spec.components** section, configure the **kserve** component as shown.

```
spec:
  components:
    kserve:
      managementState: Managed
      defaultDeploymentMode: Serverless 1
      rawDeploymentServiceConfig: Headed 2
      serving:
        ingressGateway:
          certificate:
            secretName: knative-serving-cert 3
            type: OpenshiftDefaultIngress 4
          managementState: Managed
          name: knative-serving
```

- f. Click **Save**.

The preceding configuration creates an ingress gateway for OpenShift Serverless to receive traffic from OpenShift Service Mesh. In this configuration, both KServe RawDeployment and Knative Serverless deployment modes can be used.

- 1 The configuration shown uses the default deployment mode that is selected after configuring KServe. You can set the default value when you create and deploy a model
- 2 The configuration shown uses the **Headed** mode to set the cluster to perform normal load balancing over workload replicas. For environments in which inference request load balancing is performed on the client side, set **rawDeploymentServiceConfig** to the **Headless** mode.
- 3 The configuration shown uses the default ingress certificate configured for OpenShift to secure incoming traffic to your OpenShift cluster and stores the certificate in the **knative-serving-cert** secret that is specified in the **secretName** field. The **secretName** field can only be set at the time of installation. The default value of the **secretName** field is **knative-serving-cert**. Subsequent changes to the certificate secret must be made manually. If you did not use the default **secretName** value during installation, create a new secret named **knative-serving-cert** in the **istio-system** namespace, and then restart the **istiiod-datascience-smcp-<suffix>** pod.
- 4 You can specify the following certificate types by updating the value of the **type** field:
 - **Provided**
 - **SelfSigned**
 - **OpenshiftDefaultIngress**

To use a self-signed certificate or to provide your own, update the value of the **secretName** field to specify your secret name and change the value of the **type** field to **SelfSigned** or **Provided**.



NOTE

If you provide your own certificate, the certificate must specify the domain name used by the ingress controller of your OpenShift cluster. You can check this value by running the following command:

```
$ oc get ingresses.config.openshift.io cluster -o
jsonpath='{.spec.domain}'
```

- You must set the value of the **managementState** field to **Managed** for both the **kserve** and **serving** components. Setting **kserve.managementState** to **Managed** triggers automated installation of KServe. Setting **serving.managementState** to **Managed** triggers automated installation of OpenShift Serverless. However, installation of OpenShift Serverless will *not* be triggered if **kserve.managementState** is not also set to **Managed**.

Verification

- Verify installation of KServe as follows:
 - In the web console, click **Workloads** → **Pods**.
 - From the project list, select **redhat-ods-applications**. This is the project in which OpenShift AI components are installed, including KServe.

- Confirm that the project includes a running pod for the KServe controller manager, similar to the following example:

NAME	READY	STATUS	RESTARTS	AGE
kserve-controller-manager-7fbb7bccd4-t4c5g	1/1	Running	0	22h
odh-model-controller-6c4759cc9b-cftmk	1/1	Running	0	129m
odh-model-controller-6c4759cc9b-ngj8b	1/1	Running	0	129m
odh-model-controller-6c4759cc9b-vnhq5	1/1	Running	0	129m

- (Knative Serverless mode only): Verify installation of OpenShift Service Mesh as follows:
 - In the web console, click **Workloads** → **Pods**.
 - From the project list, select **istio-system**. This is the project in which OpenShift Service Mesh is installed.
 - Confirm that there are running pods for the service mesh control plane, ingress gateway, and egress gateway. These pods have the naming patterns shown in the following example:

NAME	READY	STATUS	RESTARTS	AGE
istio-egressgateway-7c46668687-fzsqj	1/1	Running	0	22h
istio-ingressgateway-77f94d8f85-fhsp9	1/1	Running	0	22h
istiod-data-science-smcp-cc8cf9b8-2rkg4	1/1	Running	0	22h

- (Knative Serverless mode only): Verify installation of OpenShift Serverless as follows:
 - In the web console, click **Workloads** → **Pods**.
 - From the project list, select **knative-serving**. This is the project in which OpenShift Serverless is installed.
 - Confirm that there are numerous running pods in the **knative-serving** project, including activator, autoscaler, controller, and domain mapping pods, as well as pods for the Knative Istio controller (which controls the integration of OpenShift Serverless and OpenShift Service Mesh). An example is shown.

NAME	READY	STATUS	RESTARTS	AGE
activator-7586f6f744-nvdlb	2/2	Running	0	22h
activator-7586f6f744-sd77w	2/2	Running	0	22h
autoscaler-764fdf5d45-p2v98	2/2	Running	0	22h
autoscaler-764fdf5d45-x7dc6	2/2	Running	0	22h
autoscaler-hpa-7c7c4cd96d-2lkzg	1/1	Running	0	22h
autoscaler-hpa-7c7c4cd96d-gks9j	1/1	Running	0	22h
controller-5fdcf9567c-6cj9d	1/1	Running	0	22h
controller-5fdcf9567c-bf5x7	1/1	Running	0	22h
domain-mapping-56ccd85968-2hjvp	1/1	Running	0	22h
domain-mapping-56ccd85968-lg6mw	1/1	Running	0	22h
domainmapping-webhook-769b88695c-gp2hk	1/1	Running	0	22h
domainmapping-webhook-769b88695c-npn8g	1/1	Running	0	22h
net-istio-controller-7dfc6f668c-jb4xk	1/1	Running	0	22h
net-istio-controller-7dfc6f668c-jxs5p	1/1	Running	0	22h
net-istio-webhook-66d8f75d6f-bgd5r	1/1	Running	0	22h
net-istio-webhook-66d8f75d6f-hld75	1/1	Running	0	22h
webhook-7d49878bc4-8xjbr	1/1	Running	0	22h
webhook-7d49878bc4-s4xx4	1/1	Running	0	22h

6.3. MANUALLY INSTALLING KSERVE

If you have already installed the Red Hat OpenShift Service Mesh Operator and created a **ServiceMeshControlPlane** resource or if you have installed the Red Hat OpenShift Serverless Operator and created a **KNativeServing** resource, the Red Hat OpenShift AI Operator cannot install KServe and configure its dependencies. In this situation, you must install KServe manually.



IMPORTANT

The procedures in this section show how to perform a *new* installation of KServe and its dependencies and are intended as a complete installation and configuration reference. If you have already installed and configured OpenShift Service Mesh or OpenShift Serverless, you might not need to follow all steps. If you are unsure about what updates to apply to your existing configuration to use KServe, contact Red Hat Support.



NOTE

You can run KServe in **Unmanaged** mode during manual installations of the single-model serving platform. This mode is useful when you need more control over KServe components, such as modifying resource limits for the KServe controller.

6.3.1. Installing KServe dependencies

Before you install KServe, you must install and configure some dependencies. Specifically, you must create Red Hat OpenShift Service Mesh and Knative Serving instances and then configure secure gateways for Knative Serving.



NOTE

Currently, only OpenShift Service Mesh v2 is supported. For more information, see [Supported Configurations](#).

6.3.2. Creating an OpenShift Service Mesh instance

The following procedure shows how to create a Red Hat OpenShift Service Mesh instance.

Prerequisites

- You have cluster administrator privileges for your OpenShift cluster.
- Your cluster has a node with 4 CPUs and 16 GB memory.
- You have installed the OpenShift CLI (**oc**) as described in the appropriate documentation for your cluster:
 - [Installing the OpenShift CLI](#) for OpenShift Container Platform
 - [Installing the OpenShift CLI](#) for Red Hat OpenShift Service on AWS
- You have [installed](#) the Red Hat OpenShift Service Mesh Operator and dependent Operators.

Procedure

1. In a terminal window, if you are not already logged in to your OpenShift cluster as a cluster administrator, log in to the OpenShift CLI (**oc**) as shown in the following example:

```
$ oc login <openshift_cluster_url> -u <admin_username> -p <password>
```

2. Create the required namespace for Red Hat OpenShift Service Mesh.

```
$ oc create ns istio-system
```

You see the following output:

```
namespace/istio-system created
```

3. Define a **ServiceMeshControlPlane** object in a YAML file named **smcp.yaml** with the following contents:

```
apiVersion: maistra.io/v2
kind: ServiceMeshControlPlane
metadata:
  name: minimal
  namespace: istio-system
spec:
  tracing:
    type: None
  addons:
    grafana:
      enabled: false
    kiali:
      name: kiali
      enabled: false
    prometheus:
      enabled: false
    jaeger:
      name: jaeger
  security:
    dataPlane:
      mtls: true
    identity:
      type: ThirdParty
  techPreview:
    meshConfig:
      defaultConfig:
        terminationDrainDuration: 35s
  gateways:
    ingress:
      service:
        metadata:
          labels:
            knative: ingressgateway
  proxy:
    networking:
      trafficControl:
        inbound:
```

```
excludedPorts:
```

```
- 8444
```

```
- 8022
```

For more information about the values in the YAML file, see the [Service Mesh control plane configuration reference](#).

4. Create the service mesh control plane.

```
$ oc apply -f smcp.yaml
```

Verification

- Verify creation of the service mesh instance as follows:

- In the OpenShift CLI, enter the following command:

```
$ oc get pods -n istio-system
```

The preceding command lists all running pods in the **istio-system** project. This is the project in which OpenShift Service Mesh is installed.

- Confirm that there are running pods for the service mesh control plane, ingress gateway, and egress gateway. These pods have the following naming patterns:

NAME	READY	STATUS	RESTARTS	AGE
istio-egressgateway-7c46668687-fzsqj	1/1	Running	0	22h
istio-ingressgateway-77f94d8f85-fhsp9	1/1	Running	0	22h
istiod-data-science-smcp-cc8cfd9b8-2rkg4	1/1	Running	0	22h

6.3.3. Creating a Knative Serving instance

The following procedure shows how to install Knative Serving and then create an instance.

Prerequisites

- You have cluster administrator privileges for your OpenShift cluster.
- Your cluster has a node with 4 CPUs and 16 GB memory.
- You have installed the OpenShift CLI (**oc**) as described in the appropriate documentation for your cluster:
 - [Installing the OpenShift CLI](#) for OpenShift Container Platform
 - [Installing the OpenShift CLI](#) for Red Hat OpenShift Service on AWS
- You have created a Red Hat OpenShift Service Mesh instance. See [Creating an OpenShift Service Mesh instance](#).
- You have [installed](#) the Red Hat OpenShift Serverless Operator.

Procedure

1. In a terminal window, if you are not already logged in to your OpenShift cluster as a cluster administrator, log in to the OpenShift CLI (**oc**) as shown in the following example:

```
$ oc login <openshift_cluster_url> -u <admin_username> -p <password>
```

2. Check whether the required project (that is, *namespace*) for Knative Serving already exists.

```
$ oc get ns knative-serving
```

If the project exists, you see output similar to the following example:

```
NAME          STATUS AGE
knative-serving Active 4d20h
```

3. If the **knative-serving** project *doesn't* already exist, create it.

```
$ oc create ns knative-serving
```

You see the following output:

```
namespace/knative-serving created
```

4. Define a **ServiceMeshMember** object in a YAML file called **default-smm.yaml** with the following contents:

```
apiVersion: maistra.io/v1
kind: ServiceMeshMember
metadata:
  name: default
  namespace: knative-serving
spec:
  controlPlaneRef:
    namespace: istio-system
    name: minimal
```

5. Create the **ServiceMeshMember** object in the **istio-system** namespace.

```
$ oc apply -f default-smm.yaml
```

You see the following output:

```
servicemeshmember.maistra.io/default created
```

6. Define a **KnativeServing** object in a YAML file called **knativeserving-istio.yaml** with the following contents:

```
apiVersion: operator.knative.dev/v1beta1
kind: KnativeServing
metadata:
  name: knative-serving
  namespace: knative-serving
annotations:
  serverless.openshift.io/default-enable-http2: "true"
```

```

spec:
  workloads:
    - name: net-istio-controller
      env:
        - container: controller
          envVars:
            - name: ENABLE_SECRET_INFORMER_FILTERING_BY_CERT_UID
              value: 'true'
      - annotations:
          sidecar.istio.io/inject: "true" ❶
          sidecar.istio.io/rewriteAppHTTPProbers: "true" ❷
        name: activator
      - annotations:
          sidecar.istio.io/inject: "true"
          sidecar.istio.io/rewriteAppHTTPProbers: "true"
        name: autoscaler
  ingress:
    istio:
      enabled: true
  config:
    features:
      kubernetes.podspec-affinity: enabled
      kubernetes.podspec-nodeselector: enabled
      kubernetes.podspec-tolerations: enabled
      kubernetes.podspec-schedulename: enabled
      multi-container-probing: enabled

```

The preceding file defines a custom resource (CR) for a **KnativeServing** object. The CR also adds the following actions to each of the activator and autoscaler pods:

- ❶ Injects an Istio sidecar to the pod. This makes the pod part of the service mesh.
- ❷ Enables the Istio sidecar to rewrite the HTTP liveness and readiness probes for the pod.



NOTE

If you configure a custom domain for a Knative service, you can use a TLS certificate to secure the mapped service. To do this, you must create a TLS secret, and then update the **DomainMapping** CR to use the TLS secret that you have created. For more information, see [Securing a mapped service using a TLS certificate](#) in the Red Hat OpenShift Serverless documentation.

7. Optional: You can configure **knative serving** to allow zero initial scale for revisions by using the **allow-zero-initial-scale** autoscaler configuration:

```

spec:
  config:
    autoscaler:
      allow-zero-initial-scale: "true"

```

8. Optional: You can enable **initContainers** for your **knative serving** instance by adding the **kubernetes.podspec-init-containers** feature flag as shown in the following example:

```
spec:
  config:
    features:
      kubernetes.podspec-init-containers: "enabled"
```

You can also add the flag in the **config-feature** ConfigMap in the **knative-serving** namespace. For more information, see [Feature and extension flags](#).

9. Create the **KnativeServing** object in the specified **knative-serving** namespace.

```
$ oc apply -f knativeserving-istio.yaml
```

You see the following output:

```
knativeserving.operator.knative.dev/knative-serving created
```

Verification

- Review the default **ServiceMeshMemberRoll** object in the **istio-system** namespace.

```
$ oc describe smmr default -n istio-system
```

In the description of the **ServiceMeshMemberRoll** object, locate the **Status.Members** field and confirm that it includes the **knative-serving** namespace.

- Verify creation of the Knative Serving instance as follows:
 - In the OpenShift CLI, enter the following command:

```
$ oc get pods -n knative-serving
```

The preceding command lists all running pods in the **knative-serving** project. This is the project in which you created the Knative Serving instance.

- Confirm that there are numerous running pods in the **knative-serving** project, including activator, autoscaler, controller, and domain mapping pods, as well as pods for the Knative Istio controller, which controls the integration of OpenShift Serverless and OpenShift Service Mesh. An example is shown.

NAME	READY	STATUS	RESTARTS	AGE
activator-7586f6f744-nvdlb	2/2	Running	0	22h
activator-7586f6f744-sd77w	2/2	Running	0	22h
autoscaler-764fdf5d45-p2v98	2/2	Running	0	22h
autoscaler-764fdf5d45-x7dc6	2/2	Running	0	22h
autoscaler-hpa-7c7c4cd96d-2lkzg	1/1	Running	0	22h
autoscaler-hpa-7c7c4cd96d-gks9j	1/1	Running	0	22h
controller-5fdcf9567c-6cj9d	1/1	Running	0	22h
controller-5fdcf9567c-bf5x7	1/1	Running	0	22h
domain-mapping-56ccd85968-2hjvp	1/1	Running	0	22h
domain-mapping-56ccd85968-lg6mw	1/1	Running	0	22h
domainmapping-webhook-769b88695c-gp2hk	1/1	Running	0	22h
domainmapping-webhook-769b88695c-npn8g	1/1	Running	0	22h
net-istio-controller-7dfc6f668c-jb4xk	1/1	Running	0	22h
net-istio-controller-7dfc6f668c-jxs5p	1/1	Running	0	22h

net-istio-webhook-66d8f75d6f-bgd5r	1/1	Running	0	22h
net-istio-webhook-66d8f75d6f-hld75	1/1	Running	0	22h
webhook-7d49878bc4-8xjbr	1/1	Running	0	22h
webhook-7d49878bc4-s4xx4	1/1	Running	0	22h

6.3.4. Creating secure gateways for Knative Serving

To secure traffic between your Knative Serving instance and the service mesh, you must create secure gateways for your Knative Serving instance.

The following procedure shows how to use OpenSSL version 3 or later to generate a wildcard certificate and key and then use them to create local and ingress gateways for Knative Serving.



IMPORTANT

If you have your own wildcard certificate and key to specify when configuring the gateways, you can skip to step 11 of this procedure.

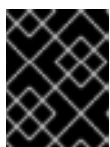
Prerequisites

- You have cluster administrator privileges for your OpenShift cluster.
- You have installed the OpenShift CLI (**oc**) as described in the appropriate documentation for your cluster:
 - [Installing the OpenShift CLI](#) for OpenShift Container Platform
 - [Installing the OpenShift CLI](#) for Red Hat OpenShift Service on AWS
- You have created a Red Hat OpenShift Service Mesh instance. See [Creating an OpenShift Service Mesh instance](#).
- You have created a Knative Serving instance. See [Creating a Knative serving instance](#).
- If you intend to generate a wildcard certificate and key, you have [downloaded and installed](#) OpenSSL version 3 or later.

Procedure

1. In a terminal window, if you are not already logged in to your OpenShift cluster as a cluster administrator, log in to the OpenShift CLI (**oc**) as shown in the following example:

```
$ oc login <openshift_cluster_url> -u <admin_username> -p <password>
```



IMPORTANT

If you have your own wildcard certificate and key to specify when configuring the gateways, skip to step 11 of this procedure.

2. Set environment variables to define base directories for generation of a wildcard certificate and key for the gateways.

```
$ export BASE_DIR=/tmp/kserve
$ export BASE_CERT_DIR=${BASE_DIR}/certs
```

- Set an environment variable to define the common name used by the ingress controller of your OpenShift cluster.

```
$ export COMMON_NAME=$(oc get ingresses.config.openshift.io cluster -o
jsonpath='{.spec.domain}' | awk -F'.' '{print $(NF-1)}.$NF')
```

- Set an environment variable to define the domain name used by the ingress controller of your OpenShift cluster.

```
$ export DOMAIN_NAME=$(oc get ingresses.config.openshift.io cluster -o
jsonpath='{.spec.domain}')
```

- Create the required base directories for the certificate generation, based on the environment variables that you previously set.

```
$ mkdir ${BASE_DIR}
$ mkdir ${BASE_CERT_DIR}
```

- Create the OpenSSL configuration for generation of a wildcard certificate.

```
$ cat <<EOF> ${BASE_DIR}/openssl-san.config
[ req ]
distinguished_name = req
[ san ]
subjectAltName = DNS:*.${DOMAIN_NAME}
EOF
```

- Generate a root certificate.

```
$ openssl req -x509 -sha256 -nodes -days 3650 -newkey rsa:2048 \
-subj "/O=Example Inc./CN=${COMMON_NAME}" \
-keyout ${BASE_CERT_DIR}/root.key \
-out ${BASE_CERT_DIR}/root.crt
```

- Generate a wildcard certificate signed by the root certificate.

```
$ openssl req -x509 -newkey rsa:2048 \
-sha256 -days 3560 -nodes \
-subj "/CN=${COMMON_NAME}/O=Example Inc." \
-extensions san -config ${BASE_DIR}/openssl-san.config \
-CA ${BASE_CERT_DIR}/root.crt \
-CAkey ${BASE_CERT_DIR}/root.key \
-keyout ${BASE_CERT_DIR}/wildcard.key \
-out ${BASE_CERT_DIR}/wildcard.crt

$ openssl x509 -in ${BASE_CERT_DIR}/wildcard.crt -text
```

- Verify the wildcard certificate.

```
$ openssl verify -CAfile ${BASE_CERT_DIR}/root.crt ${BASE_CERT_DIR}/wildcard.crt
```

- Export the wildcard key and certificate that were created by the script to new environment variables.


```
$ export TARGET_CUSTOM_CERT=${BASE_CERT_DIR}/wildcard.crt
$ export TARGET_CUSTOM_KEY=${BASE_CERT_DIR}/wildcard.key
```

11. Optional: To export *your own* wildcard key and certificate to new environment variables, enter the following commands:

```
$ export TARGET_CUSTOM_CERT=<path_to_certificate>
$ export TARGET_CUSTOM_KEY=<path_to_key>
```



NOTE

In the certificate that you provide, you must specify the domain name used by the ingress controller of your OpenShift cluster. You can check this value by running the following command:

```
$ oc get ingresses.config.openshift.io cluster -o jsonpath='{.spec.domain}'
```

12. Create a TLS secret in the **istio-system** namespace using the environment variables that you set for the wildcard certificate and key.

```
$ oc create secret tls wildcard-certs --cert=${TARGET_CUSTOM_CERT} --
key=${TARGET_CUSTOM_KEY} -n istio-system
```

13. Create a **gateways.yaml** YAML file with the following contents:

```
apiVersion: v1
kind: Service ❶
metadata:
  labels:
    experimental.istio.io/disable-gateway-port-translation: "true"
    name: knative-local-gateway
    namespace: istio-system
spec:
  ports:
    - name: http2
      port: 80
      protocol: TCP
      targetPort: 8081
  selector:
    knative: ingressgateway
  type: ClusterIP
---
apiVersion: networking.istio.io/v1beta1
kind: Gateway
metadata:
  name: knative-ingress-gateway ❷
  namespace: knative-serving
spec:
  selector:
    knative: ingressgateway
  servers:
    - hosts:
        - '*'
```

```

    port:
      name: https
      number: 443
      protocol: HTTPS
    tls:
      credentialName: wildcard-certs
      mode: SIMPLE
  ---
  apiVersion: networking.istio.io/v1beta1
  kind: Gateway
  metadata:
    name: knative-local-gateway ❸
    namespace: knative-serving
  spec:
    selector:
      knative: ingressgateway
  servers:
    - port:
        number: 8081
        name: https
        protocol: HTTPS
      tls:
        mode: ISTIO_MUTUAL
      hosts:
        - "*"

```

- ❶ Defines a service in the **istio-system** namespace for the Knative local gateway.
- ❷ Defines an ingress gateway in the **knative-serving namespace**. The gateway uses the TLS secret you created earlier in this procedure. The ingress gateway handles external traffic to Knative.
- ❸ Defines a local gateway for Knative in the **knative-serving** namespace.

14. Apply the **gateways.yaml** file to create the defined resources.

```
$ oc apply -f gateways.yaml
```

You see the following output:

```

service/knative-local-gateway created
gateway.networking.istio.io/knative-ingress-gateway created
gateway.networking.istio.io/knative-local-gateway created

```

Verification

- Review the gateways that you created.

```
$ oc get gateway --all-namespaces
```

Confirm that you see the local and ingress gateways that you created in the **knative-serving** namespace, as shown in the following example:

NAMESPACE	NAME	AGE
knative-serving	knative-ingress-gateway	69s
knative-serving	knative-local-gateway	2m

6.3.5. Installing KServe

To complete manual installation of KServe, you must install the Red Hat OpenShift AI Operator. Then, you can configure the Operator to install KServe.

Prerequisites

- You have cluster administrator privileges for your OpenShift cluster.
- Your cluster has a node with 4 CPUs and 16 GB memory.
- You have installed the OpenShift CLI (**oc**) as described in the appropriate documentation for your cluster:
 - [Installing the OpenShift CLI](#) for OpenShift Container Platform
 - [Installing the OpenShift CLI](#) for Red Hat OpenShift Service on AWS
- You have [created](#) a Red Hat OpenShift Service Mesh instance.
- You have [created](#) a Knative Serving instance.
- You have [created secure gateways](#) for Knative Serving.
- You have [installed](#) the Red Hat OpenShift AI Operator and [created](#) a **DataScienceCluster** object.

Procedure

1. Log in to the OpenShift web console as a cluster administrator.
2. In the web console, click **Operators** → **Installed Operators** and then click the Red Hat OpenShift AI Operator.
3. For installation of KServe, configure the OpenShift Service Mesh component as follows:
 - a. Click the **DSC Initialization** tab.
 - b. Click the **default-dsci** object.
 - c. Click the **YAML** tab.
 - d. In the **spec** section, add and configure the **serviceMesh** component as shown:

```
spec:
  serviceMesh:
    managementState: Unmanaged
```

- e. Click **Save**.

4. For installation of KServe, configure the KServe and OpenShift Serverless components as follows:
 - a. In the web console, click **Operators** → **Installed Operators** and then click the Red Hat OpenShift AI Operator.
 - b. Click the **Data Science Cluster** tab.
 - c. Click the **default-dsc** DSC object.
 - d. Click the **YAML** tab.
 - e. In the **spec.components** section, configure the **kserve** component as shown:

```
spec:
  components:
    kserve:
      managementState: Managed
```

- f. Within the **kserve** component, add the **serving** component, and configure it as shown:

```
spec:
  components:
    kserve:
      managementState: Managed
    serving:
      managementState: Unmanaged
```

- g. Click **Save**.

6.3.6. Configuring persistent volume claims (PVC) on KServe

Enable persistent volume claims (PVC) on your inference service so you can provision persistent storage. For more information about PVC, see [Understanding persistent storage](#).

To enable PVC, from the OpenShift AI dashboard, select the **Project** drop-down and click **knative-serving**. Then, follow the steps in [Enabling PVC support](#).

Verification

Verify that the inference service allows PVC as follows:

- In the OpenShift web console, change into the **Administrator** perspective.
- Click **Home** → **Search**.
- In **Resources**, search for **InferenceService**.
- Click the name of the inference service.
- Click the **YAML** tab.
- Confirm that **volumeMounts** appears, similar to the following output:

```
apiVersion: "serving.kserve.io/v1beta1"
kind: "InferenceService"
```

```

metadata:
  name: "sklearn-iris"
spec:
  predictor:
    model:
      runtime: kserve-mlserver
      modelFormat:
        name: sklearn
      storageUri: "gs://kfserving-examples/models/sklearn/1.0/model"
      volumeMounts:
        - name: my-dynamic-volume
          mountPath: /tmp/data
    volumes:
      - name: my-dynamic-volume
        persistentVolumeClaim:
          claimName: my-dynamic-pvc

```

6.3.7. Disabling KServe dependencies

If you have *not* enabled the KServe component (that is, you set the value of the **managementState** field to **Removed**), you must also disable the dependent Service Mesh component to avoid errors.

Prerequisites

- You have used the OpenShift CLI (**oc**) or web console to disable the KServe component.

Procedure

1. Log in to the OpenShift web console as a cluster administrator.
2. In the web console, click **Operators → Installed Operators** and then click the Red Hat OpenShift AI Operator.
3. Disable the OpenShift Service Mesh component as follows:
 - a. Click the **DSC Initialization** tab.
 - b. Click the **default-dsci** object.
 - c. Click the **YAML** tab.
 - d. In the **spec** section, add the **serviceMesh** component (if it is not already present) and configure the **managementState** field as shown:

```

spec:
  serviceMesh:
    managementState: Removed

```

- e. Click **Save**.

Verification

1. In the web console, click **Operators → Installed Operators** and then click the Red Hat OpenShift AI Operator.
The Operator details page opens.

2. In the **Conditions** section, confirm that there is no **ReconcileComplete** condition with a status value of **Unknown**.

6.4. ADDING AN AUTHORIZATION PROVIDER FOR THE SINGLE-MODEL SERVING PLATFORM

You can add [Authorino](#) as an authorization provider for the single-model serving platform. Adding an authorization provider allows you to enable token authentication for models that you deploy on the platform, which ensures that only authorized parties can make inference requests to the models.

The method that you use to add Authorino as an authorization provider depends on how you install the single-model serving platform. The installation options for the platform are described as follows:

Automated installation

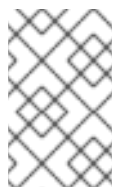
If you have not already created a **ServiceMeshControlPlane** or **KNativeServing** resource on your OpenShift cluster, you can configure the Red Hat OpenShift AI Operator to install KServe and its dependencies. You can include Authorino as part of the automated installation process.

For more information about automated installation, including Authorino, see [Configuring automated installation of KServe](#).

Manual installation

If you have already created a **ServiceMeshControlPlane** or **KNativeServing** resource on your OpenShift cluster, you *cannot* configure the Red Hat OpenShift AI Operator to install KServe and its dependencies. In this situation, you must install KServe manually. You must also manually configure Authorino.

For more information about manual installation, including Authorino, see [Manually installing KServe](#).



NOTE

You can run KServe in **Unmanaged** mode during manual installations of the single-model serving platform. This mode is useful when you need more control over KServe components, such as modifying resource limits for the KServe controller.

6.4.1. Manually adding an authorization provider

You can add [Authorino](#) as an authorization provider for the single-model serving platform. Adding an authorization provider allows you to enable token authentication for models that you deploy on the platform, which ensures that only authorized parties can make inference requests to the models.

To manually add Authorino as an authorization provider, you must install the **Red Hat - Authorino** Operator, create an Authorino instance, and then configure the OpenShift Service Mesh and KServe components to use the instance.



IMPORTANT

To manually add an authorization provider, you must make configuration updates to your OpenShift Service Mesh instance. To ensure that your OpenShift Service Mesh instance remains in a supported state, make *only* the updates shown in this section.

Prerequisites

- You have reviewed the options for adding Authorino as an authorization provider and identified manual installation as the appropriate option. See [Adding an authorization provider](#).
- You have manually installed KServe and its dependencies, including OpenShift Service Mesh. See [Manually installing KServe](#).
- When you manually installed KServe, you set the value of the **managementState** field for the **serviceMesh** component to **Unmanaged**. This setting is required for manually adding Authorino. See [Installing KServe](#).

6.4.2. Installing the Red Hat Authorino Operator

Before you can add Authorino as an authorization provider, you must install the **Red Hat - Authorino** Operator on your OpenShift cluster.

Prerequisites

- You have cluster administrator privileges for your OpenShift cluster.

Procedure

1. Log in to the OpenShift web console as a cluster administrator.
2. In the web console, click **Operators** → **OperatorHub**.
3. On the **OperatorHub** page, in the **Filter by keyword** field, type **Red Hat - Authorino**.
4. Click the **Red Hat - Authorino** Operator.
5. On the **Red Hat - Authorino** Operator page, review the Operator information and then click **Install**.
6. On the **Install Operator** page, keep the default values for **Installation mode**, **Installed Namespace** and **Update Approval**.
 - a. For **Update channel**, select **Stable**.
 - b. For **Version**, select 1.2.1 or later.
7. Click **Install**.

Verification

- In the OpenShift web console, click **Operators** → **Installed Operators** and confirm that the **Red Hat - Authorino** Operator shows one of the following statuses:
 - **Installing** - installation is in progress; wait for this to change to **Succeeded**. This might take several minutes.
 - **Succeeded** - installation is successful.

6.4.3. Creating an Authorino instance

When you have installed the **Red Hat - Authorino** Operator on your OpenShift cluster, you must create an Authorino instance.

Prerequisites

- You have installed the **Red Hat - Authorino** Operator.
- You have privileges to add resources to the project in which your OpenShift Service Mesh instance was created. See [Creating an OpenShift Service Mesh instance](#) .
For more information about OpenShift permissions, see [Using RBAC to define and apply permissions](#).

Procedure

1. Open a new terminal window.
2. Log in to the OpenShift CLI (**oc**) as follows:

```
$ oc login <openshift_cluster_url> -u <username> -p <password>
```

3. Create a namespace to install the Authorino instance.

```
$ oc new-project <namespace_for_authorino_instance>
```



NOTE

The automated installation process creates a namespace called **redhat-ods-applications-auth-provider** for the Authorino instance. Consider using the same namespace name for the manual installation.

4. To enroll the new namespace for the Authorino instance in your existing OpenShift Service Mesh instance, create a new YAML file with the following contents:

```
apiVersion: maistra.io/v1
kind: ServiceMeshMember
metadata:
  name: default
  namespace: <namespace_for_authorino_instance>
spec:
  controlPlaneRef:
    namespace: <namespace_for_service_mesh_instance>
    name: <name_of_service_mesh_instance>
```

5. Save the YAML file.
6. Create the **ServiceMeshMember** resource on your cluster.

```
$ oc create -f <file_name>.yaml
```

7. To configure an Authorino instance, create a new YAML file as shown in the following example:

```
apiVersion: operator.authorino.kuadrant.io/v1beta1
kind: Authorino
metadata:
  name: authorino
  namespace: <namespace_for_authorino_instance>
```



```
spec:
  authConfigLabelSelectors: security.opendatahub.io/authorization-group=default
  clusterWide: true
  listener:
    tls:
      enabled: false
  oidcServer:
    tls:
      enabled: false
```

8. Save the YAML file.
9. Create the **Authorino** resource on your cluster.

```
$ oc create -f <file_name>.yaml
```

10. Patch the Authorino deployment to inject an Istio sidecar, which makes the Authorino instance part of your OpenShift Service Mesh instance.

```
$ oc patch deployment <name_of_authorino_instance> -n
<namespace_for_authorino_instance> -p '{"spec": {"template": {"metadata": {"labels":
{"sidecar.istio.io/inject": "true"}}}} }'
```

Verification

- Confirm that the Authorino instance is running as follows:
 1. Check the pods (and containers) that are running in the namespace that you created for the Authorino instance, as shown in the following example:

```
$ oc get pods -n redhat-ods-applications-auth-provider -o="custom-
columns=NAME:.metadata.name,STATUS:.status.phase,CONTAINERS:.spec.containers[*]
.name"
```

2. Confirm that the output resembles the following example:

```
NAME                                STATUS  CONTAINERS
authorino-6bc64bd667-kn28z  Running  authorino,istio-proxy
```

As shown in the example, there is a single running pod for the Authorino instance. The pod has containers for Authorino and for the Istio sidecar that you injected.

6.4.4. Configuring an OpenShift Service Mesh instance to use Authorino

When you have created an Authorino instance, you must configure your OpenShift Service Mesh instance to use Authorino as an authorization provider.



IMPORTANT

To ensure that your OpenShift Service Mesh instance remains in a supported state, make *only* the configuration updates shown in the following procedure.

Prerequisites

- You have created an Authorino instance and enrolled the namespace for the Authorino instance in your OpenShift Service Mesh instance.
- You have privileges to modify the OpenShift Service Mesh instance. See [Creating an OpenShift Service Mesh instance](#).

Procedure

1. In a terminal window, if you are not already logged in to your OpenShift cluster as a user that has privileges to update the OpenShift Service Mesh instance, log in to the OpenShift CLI (**oc**) as shown in the following example:

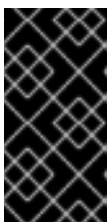
```
$ oc login <openshift_cluster_url> -u <username> -p <password>
```

2. Create a new YAML file with the following contents:

```
spec:
  techPreview:
    meshConfig:
      extensionProviders:
        - name: redhat-ods-applications-auth-provider
      envoyExtAuthzGrpc:
        service: <name_of_authorino_instance>-authorino-
        authorization.<namespace_for_authorino_instance>.svc.cluster.local
        port: 50051
```

3. Save the YAML file.
4. Use the **oc patch** command to apply the YAML file to your OpenShift Service Mesh instance.

```
$ oc patch smcp <name_of_service_mesh_instance> --type merge -n
<namespace_for_service_mesh_instance> --patch-file <file_name>.yaml
```



IMPORTANT

You can apply the configuration shown as a patch *only* if you have not already specified other extension providers in your OpenShift Service Mesh instance. If you have already specified other extension providers, you must manually edit your **ServiceMeshControlPlane** resource to add the configuration.

Verification

- Verify that your Authorino instance has been added as an extension provider in your OpenShift Service Mesh configuration as follows:
 1. Inspect the **ConfigMap** object for your OpenShift Service Mesh instance:

```
$ oc get configmap istio-<name_of_service_mesh_instance> -n
<namespace_for_service_mesh_instance> --output=jsonpath={.data.mesh}
```

2. Confirm that you see output similar to the following example, which shows that the Authorino instance has been successfully added as an extension provider.

```
defaultConfig:
```

```

discoveryAddress: istiod-data-science-smcp.istio-system.svc:15012
proxyMetadata:
  ISTIO_META_DNS_AUTO_ALLOCATE: "true"
  ISTIO_META_DNS_CAPTURE: "true"
  PROXY_XDS_VIA_AGENT: "true"
terminationDrainDuration: 35s
tracing: {}
dnsRefreshRate: 300s
enablePrometheusMerge: true
extensionProviders:
- envoyExtAuthzGrpc:
    port: 50051
    service: authorino-authorino-authorization.opendatahub-auth-provider.svc.cluster.local
    name: opendatahub-auth-provider
ingressControllerMode: "OFF"
rootNamespace: istio-system
trustDomain: null%

```

6.4.5. Configuring authorization for KServe

To configure the single-model serving platform to use Authorino, you must create a global **AuthorizationPolicy** resource that is applied to the KServe predictor pods that are created when you deploy a model. In addition, to account for the multiple network hops that occur when you make an inference request to a model, you must create an **EnvoyFilter** resource that continually resets the HTTP host header to the one initially included in the inference request.

Prerequisites

- You have created an Authorino instance and configured your OpenShift Service Mesh to use it.
- You have privileges to update the KServe deployment on your cluster.
- You have privileges to add resources to the project in which your OpenShift Service Mesh instance was created. See [Creating an OpenShift Service Mesh instance](#).

Procedure

1. In a terminal window, if you are not already logged in to your OpenShift cluster as a user that has privileges to update the KServe deployment, log in to the OpenShift CLI (**oc**) as shown in the following example:

```
$ oc login <openshift_cluster_url> -u <username> -p <password>
```

2. Create a new YAML file with the following contents:

```

apiVersion: security.istio.io/v1beta1
kind: AuthorizationPolicy
metadata:
  name: kserve-predictor
spec:
  action: CUSTOM
  provider:
    name: redhat-ods-applications-auth-provider 1
  rules:

```

```

- to:
  - operation:
      notPaths:
        - /healthz
        - /debug/pprof/
        - /metrics
        - /wait-for-drain
selector:
  matchLabels:
    component: predictor

```

- 1 The name that you specify must match the name of the extension provider that you added to your OpenShift Service Mesh instance.

3. Save the YAML file.
4. Create the **AuthorizationPolicy** resource in the namespace for your OpenShift Service Mesh instance.

```
$ oc create -n <namespace_for_service_mesh_instance> -f <file_name>.yaml
```

5. Create another new YAML file with the following contents:

```

apiVersion: networking.istio.io/v1alpha3
kind: EnvoyFilter
metadata:
  name: activator-host-header
spec:
  priority: 20
  workloadSelector:
    labels:
      component: predictor
  configPatches:
    - applyTo: HTTP_FILTER
      match:
        listener:
          filterChain:
            filter:
              name: envoy.filters.network.http_connection_manager
      patch:
        operation: INSERT_BEFORE
        value:
          name: envoy.filters.http.lua
          typed_config:
            '@type': type.googleapis.com/envoy.extensions.filters.http.lua.v3.Lua
            inlineCode: |
              function envoy_on_request(request_handle)
                local headers = request_handle:headers()
                if not headers then
                  return
                end
                local original_host = headers:get("k-original-host")
                if original_host then
                  port_seperator = string.find(original_host, ":", 7)
                  if port_seperator then

```

```

        original_host = string.sub(original_host, 0, port_seperator-1)
      end
      headers:replace('host', original_host)
    end
  end
end

```

The **EnvoyFilter** resource shown continually resets the HTTP host header to the one initially included in any inference request.

6. Create the **EnvoyFilter** resource in the namespace for your OpenShift Service Mesh instance.

```
$ oc create -n <namespace_for_service_mesh_instance> -f <file_name>.yaml
```

Verification

- Check that the **AuthorizationPolicy** resource was successfully created.

```
$ oc get authorizationpolicies -n <namespace_for_service_mesh_instance>
```

Confirm that you see output similar to the following example:

```

NAME          AGE
kserve-predictor 28h

```

- Check that the **EnvoyFilter** resource was successfully created.

```
$ oc get envoyfilter -n <namespace_for_service_mesh_instance>
```

Confirm that you see output similar to the following example:

```

NAME                      AGE
activator-host-header      28h

```

CHAPTER 7. INSTALLING THE MULTI-MODEL SERVING PLATFORM

For deploying small and medium-sized models, OpenShift AI includes a *multi-model serving platform* that is based on the ModelMesh component. On the multi-model serving platform, multiple models can be deployed from the same model server and share the server resources.



IMPORTANT

Starting with OpenShift AI version 2.19, the multi-model serving platform based on ModelMesh is deprecated. You can continue to deploy models on the multi-model serving platform, but it is recommended that you migrate to the single-model serving platform.

For more information or for help on using the single-model serving platform, contact your account manager.

To install the multi-model serving platform or ModelMesh, follow the steps described in [Installing Red Hat OpenShift AI components by using the CLI](#) or [Installing Red Hat OpenShift AI components by using the web console](#).

CHAPTER 8. ACCESSING THE DASHBOARD

After you have installed OpenShift AI and added users, you can access the URL for your OpenShift AI console and share the URL with the users to let them log in and work on their models.

Prerequisites

- You have installed OpenShift AI on your OpenShift cluster.
- You have added at least one user to the user group for OpenShift AI.

Procedure

1. Log in to OpenShift web console.
2. Click the application launcher ().
3. Right-click on **Red Hat OpenShift AI** and copy the URL for your OpenShift AI instance.
4. Provide this instance URL to your data scientists to let them log in to OpenShift AI.

Verification

- Confirm that you and your users can log in to OpenShift AI by using the instance URL.

Note: In the Red Hat OpenShift AI dashboard, users can view the list of the installed OpenShift AI components, their corresponding source (upstream) components, and the versions of the installed components, as described in [Viewing installed components](#).

Additional resources

- [Logging in to OpenShift AI](#)
- [Adding users to OpenShift AI user groups](#)

CHAPTER 9. ENABLING ACCELERATORS

Before you can use an accelerator in OpenShift AI, you must install the relevant software components. The installation process varies based on the accelerator type.

Prerequisites

- You have logged in to your OpenShift cluster.
- You have the **cluster-admin** role in your OpenShift cluster.
- You have installed an accelerator and confirmed that it is detected in your environment.

Procedure

1. Follow the appropriate documentation to enable your accelerator:
 - **NVIDIA GPUs:** See [Enabling NVIDIA GPUs](#).
 - **Intel Gaudi AI accelerators:** See [Enabling Intel Gaudi AI accelerators](#).
 - **AMD GPUs:** See [Enabling AMD GPUs](#).
2. After installing your accelerator, create an accelerator profile as described in: [Working with accelerator profiles](#).



IMPORTANT

By default, hardware profiles are hidden in the dashboard navigation menu and user interface, while accelerator profiles remain visible. In addition, user interface components associated with the deprecated accelerator profiles functionality are still displayed. To show the **Settings → Hardware profiles** option in the dashboard navigation menu, and the user interface components associated with hardware profiles, set the **disableHardwareProfiles** value to **false** in the **Odhdashboardconfig** custom resource (CR) in OpenShift. For more information about setting dashboard configuration options, see [Customizing the dashboard](#).

Verification

- From the **Administrator** perspective, go to the **Operators → Installed Operators** page. Confirm that the following Operators appear:
 - The Operator for your accelerator
 - Node Feature Discovery (NFD)
 - Kernel Module Management (KMM)
- The accelerator is correctly detected a few minutes after full installation of the Node Feature Discovery (NFD) and the relevant accelerator Operator. The OpenShift CLI (**oc**) displays the appropriate output for the GPU worker node. For example, here is output confirming that an NVIDIA GPU is detected:

```
# Expected output when the accelerator is detected correctly
```



```
oc describe node <node name>
...
Capacity:
  cpu:          4
  ephemeral-storage: 313981932Ki
  hugepages-1Gi:  0
  hugepages-2Mi:  0
  memory:        16076568Ki
  nvidia.com/gpu: 1
  pods:          250
Allocatable:
  cpu:          3920m
  ephemeral-storage: 288292006229
  hugepages-1Gi:  0
  hugepages-2Mi:  0
  memory:        12828440Ki
  nvidia.com/gpu: 1
  pods:          250
```

CHAPTER 10. WORKING WITH CERTIFICATES

When you install Red Hat OpenShift AI, OpenShift automatically applies a default Certificate Authority (CA) bundle to manage authentication for most OpenShift AI components, such as workbenches and model servers. These certificates are trusted self-signed certificates that help secure communication. However, as a cluster administrator, you might need to configure additional self-signed certificates to use some components, such as the data science pipeline server and object storage solutions. If an OpenShift AI component uses a self-signed certificate that is not part of the existing cluster-wide CA bundle, you have the following options for including the certificate:

- Add it to the OpenShift cluster-wide CA bundle.
- Add it to a custom CA bundle, separate from the cluster-wide CA bundle.

As a cluster administrator, you can also change how to manage authentication for OpenShift AI as follows:

- Manually manage certificate changes, instead of relying on the OpenShift AI Operator to handle them automatically.
- Remove the cluster-wide CA bundle, either from all namespaces or specific ones. If you prefer to implement a different authentication approach, you can override the default OpenShift AI behavior, as described in *Removing the CA bundle*.

10.1. UNDERSTANDING HOW OPENSIFT AI HANDLES CERTIFICATES

After installing OpenShift AI, the Red Hat OpenShift AI Operator automatically creates an empty **odh-trusted-ca-bundle** configuration file (ConfigMap). The Cluster Network Operator (CNO) injects the cluster-wide CA bundle into the **odh-trusted-ca-bundle** configMap with the label **"config.openshift.io/inject-trusted-cabundle"**.

```
apiVersion: v1
kind: ConfigMap
metadata:
  labels:
    app.kubernetes.io/part-of: opendatahub-operator
    config.openshift.io/inject-trusted-cabundle: 'true'
name: odh-trusted-ca-bundle
```

After the CNO operator injects the bundle, it updates the ConfigMap with the contents of the **ca-bundle.crt** file.

```
apiVersion: v1
kind: ConfigMap
metadata:
  labels:
    app.kubernetes.io/part-of: opendatahub-operator
    config.openshift.io/inject-trusted-cabundle: 'true'
name: odh-trusted-ca-bundle
data:
  ca-bundle.crt: |
    <BUNDLE OF CLUSTER-WIDE CERTIFICATES>
```

The management of CA bundles is configured through the Data Science Cluster Initialization (DSCI) object. Within this object, you can set the **spec.trustedCABundle.managementState** field to one of the following values:

- **Managed:** (Default) The Red Hat OpenShift AI Operator manages the **odh-trusted-ca-bundle** ConfigMap and adds it to all non-reserved existing and new namespaces. It does not add the ConfigMap to any reserved or system namespaces, such as **default**, **openshift-*** or **kube-***. The Red Hat OpenShift AI Operator automatically updates the ConfigMap to reflect any changes made to the **customCABundle** field.
- **Unmanaged:** The Red Hat OpenShift AI administrator manually manages the **odh-trusted-ca-bundle** ConfigMap, instead of allowing the Operator to manage it. Changing the **managementState** from **Managed** to **Unmanaged** does not remove the **odh-trusted-ca-bundle** ConfigMap. However, the ConfigMap is no longer automatically updated if changes are made to the **customCABundle** field.

The **Unmanaged** setting is useful if your organization implements a different method for managing trusted CA bundles, such as Ansible automation, and does not want the Red Hat OpenShift AI Operator to handle certificates automatically. This setting provides greater control, preventing the Operator from overwriting custom configurations.

- **Removed:** The Red Hat OpenShift AI Operator removes the **odh-trusted-ca-bundle** ConfigMap, if present, and prevents ConfigMaps from being created in new namespaces. Changing this field from **Managed** to **Removed** also deletes the ConfigMap from existing namespaces. This is the default value after upgrading Red Hat OpenShift AI from 2.7 or earlier versions to 2.25.

The **Removed** setting reduces complexity and mitigates security risks, such as unauthorized certificate changes. In high-security environments, removing the CA bundle ensures that only approved CAs are trusted, reducing the risk of cyberattacks. For example, your organization might want to restrict cluster administrators from creating trusted CA bundles to prevent OpenShift pods from communicating externally.

10.2. ADDING CERTIFICATES

If you must use a self-signed certificate that is not part of the existing cluster-wide CA bundle, you have two options for configuring the certificate:

- Add it to the cluster-wide CA bundle.
This option is useful when the certificate is needed for secure communication across multiple services or when it's required by security policies to be trusted cluster-wide. This option ensures that all services and components in the cluster trust the certificate automatically. It simplifies management because the certificate is trusted across the entire cluster, avoiding the need to configure the certificate separately for each service.
- Add it to a custom CA bundle that is separate from the OpenShift cluster-wide bundle. Consider this option for the following scenarios:
 - Limit scope: Only specific services need the certificate, not the whole cluster.
 - Isolation: Keeps custom certificates separate, preventing changes to the global configuration.
 - Avoid global impact: Does not affect services that do not need the certificate.
 - Easier management: Makes it simpler to manage certificates for specific services.

10.3. ADDING CERTIFICATES TO A CLUSTER-WIDE CA BUNDLE

You can add a self-signed certificate to a cluster-wide Certificate Authority (CA) bundle (**ca-bundle.crt**).

When the cluster-wide CA bundle is updated, the Cluster Network Operator (CNO) automatically detects the change and injects the updated bundle into the **odh-trusted-ca-bundle** ConfigMap, making the certificate available to OpenShift AI components.

Note: By default, the management state for the Trusted CA bundle is **Managed** (that is, the **spec.trustedCABundle.managementState** field in the Red Hat OpenShift AI Operator's DSCI object is set to **Managed**). If you change this setting to **Unmanaged**, you must manually update the **odh-trusted-ca-bundle** ConfigMap to include the updated cluster-wide CA bundle.

Alternatively, you can add certificates to a custom CA bundle, as described in [Adding certificates to a custom CA bundle](#).

Prerequisites

- You have created a self-signed certificate and saved the certificate to a file. For example, you have created a certificate using OpenSSL and saved it to a file named **example-ca.crt**.
- You have cluster administrator access for the OpenShift cluster where Red Hat OpenShift AI is installed.
- You have installed the OpenShift CLI (**oc**) as described in the appropriate documentation for your cluster:
 - [Installing the OpenShift CLI](#) for OpenShift Container Platform
 - [Installing the OpenShift CLI](#) for Red Hat OpenShift Service on AWS

Procedure

1. Create a ConfigMap that includes the root CA certificate used to sign the certificate, where **</path/to/example-ca.crt>** is the path to the CA certificate bundle on your local file system:

```
oc create configmap custom-ca \
  --from-file=ca-bundle.crt=</path/to/example-ca.crt> \
  -n openshift-config
```

2. Update the cluster-wide proxy configuration with the newly-created ConfigMap:

```
oc patch proxy/cluster \
  --type=merge \
  --patch='{ "spec": { "trustedCA": { "name": "custom-ca" } } }'
```

Verification

Run the following command to verify that all non-reserved namespaces contain the **odh-trusted-ca-bundle** ConfigMap:

```
oc get configmaps --all-namespaces -l app.kubernetes.io/part-of=opendatahub-operator | grep odh-trusted-ca-bundle
```

Additional resources

- [Configuring certificates](#) in the OpenShift Container Platform documentation
- [Injecting a custom CA Bundle](#) in the Red Hat OpenShift Service on AWS documentation
- [Injecting a custom CA Bundle](#) in the OpenShift Dedicated documentation

10.4. ADDING CERTIFICATES TO A CUSTOM CA BUNDLE

You can add self-signed certificates to a custom CA bundle that is separate from the OpenShift cluster-wide bundle.

This method is ideal for scenarios where components need access to external resources that require a self-signed certificate. For example, you may need to add self-signed certificates to grant data science pipelines access S3-compatible object storage.

Prerequisites

- You have created a self-signed certificate and saved the certificate to a file. For example, you have created a certificate using OpenSSL and saved it to a file named **example-ca.crt**.
- You have cluster administrator access for the OpenShift cluster where Red Hat OpenShift AI is installed.
- You have installed the OpenShift CLI (**oc**) as described in the appropriate documentation for your cluster:
 - [Installing the OpenShift CLI](#) for OpenShift Container Platform
 - [Installing the OpenShift CLI](#) for Red Hat OpenShift Service on AWS

Procedure

1. Log in to OpenShift.
2. Click **Operators** → **Installed Operators** and then click the Red Hat OpenShift AI Operator.
3. Click the **DSC Initialization** tab.
4. Click the **default-dsci** object.
5. Click the **YAML** tab.
6. In the **spec.trustedCABundle** section, add the custom certificate to the **customCABundle** field, as shown in the following example:

```
spec:
  trustedCABundle:
    managementState: Managed
    customCABundle: |
      -----BEGIN CERTIFICATE-----
      examplebundle123
      -----END CERTIFICATE-----
```

7. Click **Save**.

The Red Hat OpenShift AI Operator automatically updates the ConfigMap to reflect any changes made to the **customCABundle** field. It adds the **odh-ca-bundle.crt** file containing the certificates to the **odh-trusted-ca-bundle** ConfigMap, as shown in the following example:

```
apiVersion: v1
kind: ConfigMap
metadata:
  labels:
    app.kubernetes.io/part-of: opendatahub-operator
    config.openshift.io/inject-trusted-cabundle: 'true'
  name: odh-trusted-ca-bundle
data:
  ca-bundle.crt: |
    <BUNDLE OF CLUSTER-WIDE CERTIFICATES>
  odh-ca-bundle.crt: |
    <BUNDLE OF CUSTOM CERTIFICATES>
```

Verification

Run the following command to verify that a non-reserved namespace contains the **odh-trusted-ca-bundle** ConfigMap and that the ConfigMap contains your **customCABundle** value. In the following command, *example-namespace* is the non-reserved namespace and *examplebundle123* is the **customCABundle** value.

```
oc get configmap odh-trusted-ca-bundle -n example-namespace -o yaml | grep examplebundle123
```

10.5. USING SELF-SIGNED CERTIFICATES WITH OPENSIFT AI COMPONENTS

Some OpenShift AI components have additional options or required configuration for self-signed certificates.

10.5.1. Accessing S3-compatible object storage with self-signed certificates

To securely connect OpenShift AI components to object storage solutions or databases that are deployed within an OpenShift cluster that uses self-signed certificates, you must provide a certificate authority (CA) certificate. Each namespace includes a ConfigMap named **kube-root-ca.crt**, which contains the CA certificate of the internal API Server.

Prerequisites

- You have cluster administrator privileges for your OpenShift cluster.
- You have installed the OpenShift CLI (**oc**) as described in the appropriate documentation for your cluster:
 - [Installing the OpenShift CLI](#) for OpenShift Container Platform
 - [Installing the OpenShift CLI](#) for Red Hat OpenShift Service on AWS
- You have deployed an object storage solution or database in your OpenShift cluster.

Procedure

1. In a terminal window, log in to the OpenShift CLI (**oc**) as shown in the following example:

```
oc login api.<cluster_name>.<cluster_domain>:6443 --web
```

2. Retrieve the current OpenShift AI trusted CA configuration and store it in a new file:

```
oc get dscinitializations.dscinitialization.opendatahub.io default-dsci -o json | jq -r  
'spec.trustedCABundle.customCABundle' > /tmp/my-custom-ca-bundles.crt
```

3. Add the cluster's **kube-root-ca.crt** ConfigMap to the OpenShift AI trusted CA configuration:

```
oc get configmap kube-root-ca.crt -o jsonpath="{['data']['ca.crt']}" >> /tmp/my-custom-ca-bundles.crt
```

4. Update the OpenShift AI trusted CA configuration to trust certificates issued by the certificate authorities in **kube-root-ca.crt**:

```
oc patch dscinitialization default-dsci --type='json' -  
p=[{"op":"replace","path":"/spec/trustedCABundle/customCABundle","value":""$(awk '{printf  
"%s\\n", $0}' /tmp/my-custom-ca-bundles.crt)"}]
```

Verification

- You can successfully deploy components that are configured to use object storage solutions or databases that are deployed in the OpenShift cluster. For example, a pipeline server that is configured to use a database deployed in the cluster starts successfully.



NOTE

You can verify your new certificate configuration by following the steps in the *OpenShift AI tutorial – Fraud Detection example*. Run the script to install local object storage buckets and create connections, and then enable data science pipelines.

For more information about running the script to install local object storage buckets, see [Running a script to install local object storage buckets and create connections](#).

For more information about enabling data science pipelines, see [Enabling data science pipelines](#).

10.5.2. Configuring a certificate for data science pipelines

By default, OpenShift AI includes OpenShift cluster-wide certificates in the **odh-trusted-ca-bundle** ConfigMap. These cluster-wide certificates cover most components, such as workbenches and model servers. However, the pipeline server might require additional Certificate Authority (CA) configuration, especially when interacting with external systems that use self-signed or custom certificates.

You have the following options for adding the certificate for data science pipelines:

- Add them to the cluster-wide CA bundle, as described in [Adding certificates to a cluster-wide CA bundle](#).
- Add them to a custom bundle as described in [Adding certificates to a custom CA bundle](#).

- Provide a CA bundle that is only used for data science pipelines, as described in the following procedure.

Prerequisites

- You have cluster administrator access for the OpenShift cluster where Red Hat OpenShift AI is installed.
- You have created a self-signed certificate and saved the certificate to a file. For example, you have created a certificate using OpenSSL and saved it to a file named **example-ca.crt**.
- You have configured a data science pipeline server.

Procedure

1. Log in to the OpenShift console.
2. From **Workloads → ConfigMaps**, create a ConfigMap with the required bundle in the same data science project as the target data science pipeline:

```
kind: ConfigMap
apiVersion: v1
metadata:
  name: custom-ca-bundle
data:
  ca-bundle.crt: |
    # contents of ca-bundle.crt
```

3. Add the following snippet to the **.spec.apiserver.caBundle** field of the underlying Data Science Pipelines Application (DSPA):

```
apiVersion: datasciencepipelinesapplications.opendatahub.io/v1
kind: DataSciencePipelinesApplication
metadata:
  name: data-science-dspa
spec:
  ...
  apiServer:
  ...
  cABundle:
    configMapName: custom-ca-bundle
    configMapKey: ca-bundle.crt
```

4. Save the ConfigMap. The pipeline server pod automatically redeploys with the updated bundle.

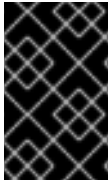
Verification

Confirm that your CA bundle was successfully mounted:

1. Log in to the OpenShift console.
2. Go to the data science project that has the target data science pipeline.
3. Click the **Pods** tab.
4. Click the pipeline server pod with the **ds-pipeline-dspa-`<hash>`** prefix.

5. Click **Terminal**.
6. Enter **cat /dsp-custom-certs/dsp-ca.crt**.
7. Verify that your CA bundle is present within this file.

10.5.3. Configuring a certificate for workbenches



IMPORTANT

By default, self-signed certificates apply to workbenches that you create after configuring cluster-wide certificates. To apply cluster-wide certificates to an existing workbench, stop and then restart the workbench.

Self-signed certificates are stored in **/etc/pki/tls/custom-certs/ca-bundle.crt**. Workbenches use a preset environment variable that many popular HTTP client packages point to for certificates. For packages that are not included by default, you can provide this certificate path. For example, for the **kfp** package to connect to the data science pipeline server:

```
from kfp.client import Client

with open(sa_token_file_path, 'r') as token_file:
    bearer_token = token_file.read()

client = Client(
    host='https://<GO_TO_ROUTER_OF_DS_PROJECT>',
    existing_token=bearer_token,
    ssl_ca_cert='/etc/pki/tls/custom-certs/ca-bundle.crt'
)
print(client.list_experiments())
```

10.5.4. Using the cluster-wide CA bundle for the single-model serving platform

By default, the single-model serving platform in OpenShift AI uses a self-signed certificate generated at installation for the endpoints that are created when deploying a server.

If you have configured cluster-wide certificates on your OpenShift cluster, they are used by default for other types of endpoints, such as endpoints for routes.

The following procedure explains how to use the same certificate that you already have for your OpenShift cluster.

Prerequisites

- You have cluster administrator access for the OpenShift cluster where Red Hat OpenShift AI is installed.
- You have configured cluster-wide certificates in OpenShift.
- You have configured the single-model serving platform, as described in [Installing the single-model serving platform](#).

Procedure

1. Log in to the OpenShift console.
2. From the list of projects, open the **openshift-ingress** project.
3. Click **YAML**.
4. Search for "cert" to find a secret with a name that includes "cert". For example, **rhods-internal-primary-cert-bundle-secret**. The contents of the secret should contain two items that are used for all OpenShift Routes: **tls.cert** (the certificate) and **tls.key** (the key).
5. Copy the reference to the secret.
6. From the list of projects, open the **istio-system** project.
7. Create a YAML file and paste the reference to the secret that you copied from the **openshift-ingress** YAML file.
8. Edit the YAML code to keep only the relevant content, as shown in the following example. Replace **rhods-internal-primary-cert-bundle-secret** with the name of your secret:

```
kind: Secret
apiVersion: v1
metadata:
  name: rhods-internal-primary-cert-bundle-secret
data:
  tls.crt: >-
    LS0tLS1CRUd...
  tls.key: >-
    LS0tLS1CRUd...
type: kubernetes.io/tls
```

9. Save the YAML file in the **istio-system** project.
10. Navigate to **Operators → Installed Operators → Red Hat OpenShift AI**.
11. Click **Data Science Cluster***, and then click **default-dsc → YAML**.
12. Edit the **kserve** configuration section to refer to your secret as shown in the following example. Replace **rhods-internal-primary-cert-bundle-secret** with the name of the secret that you created in Step 8.

```
kserve:
  devFlags: {}
  managementState: Managed
  serving:
    ingressGateway:
      certificate:
        secretName: rhods-internal-primary-cert-bundle-secret
        type: Provided
      managementState: Managed
      name: knative-serving
```

10.6. MANAGING CERTIFICATES WITHOUT THE RED HAT OPENSIFT AI OPERATOR

By default, the Red Hat OpenShift AI Operator manages the **odh-trusted-ca-bundle** ConfigMap, which contains the trusted CA bundle and is applied to all non-reserved namespaces in the cluster. The Operator automatically updates this ConfigMap whenever changes are made to the CA bundle.

If your organization prefers to manage trusted CA bundles independently, for example, by using Ansible automation, you can disable this default behavior to prevent automatic updates by the Red Hat OpenShift AI Operator.

Prerequisites

- You have cluster administrator privileges for your OpenShift cluster.

Procedure

1. In the OpenShift web console, click **Operators → Installed Operators** and then click the **Red Hat OpenShift AI Operator**.
2. Click the **DSC Initialization** tab.
3. Click the **default-dsci** object.
4. Click the **YAML** tab.
5. In the **spec** section, change the value of the **managementState** field for **trustedCABundle** to **Unmanaged**, as shown:

```
spec:
  trustedCABundle:
    managementState: Unmanaged
```

6. Click **Save**.
Changing the **managementState** from **Managed** to **Unmanaged** prevents automatic updates when the **customCABundle** field is modified, but does not remove the **odh-trusted-ca-bundle** ConfigMap.

Verification

1. In the **spec** section, set the value of the **customCABundle** field for **trustedCABundle**, for example:

```
spec:
  trustedCABundle:
    managementState: Unmanaged
    customCABundle: example123
```

2. Click **Save**.
3. Click **Workloads → ConfigMaps**.
4. Select a project from the project list.
5. Click the **odh-trusted-ca-bundle** ConfigMap.
6. Click the **YAML** tab and verify that the value of the **customCABundle** field did not update.

10.7. REMOVING THE CA BUNDLE

If you prefer to implement a different authentication approach for your OpenShift AI installation, you can override the default behavior by removing the CA bundle.

You have two options for removing the CA bundle:

- Remove the CA bundle from all non-reserved projects in OpenShift AI.
- Remove the CA bundle from a specific project.

10.7.1. Removing the CA bundle from all namespaces

You can remove a Certificate Authority (CA) bundle from all non-reserved namespaces in OpenShift AI. This process changes the default configuration and disables the creation of the **odh-trusted-ca-bundle** configuration file (ConfigMap), as described in [Working with certificates](#) (OpenShift AI Self-Managed) or [Working with certificates](#) (OpenShift AI Self-Managed in a disconnected environment).



NOTE

The **odh-trusted-ca-bundle** ConfigMaps are only deleted from namespaces when you set the **managementState** of **trustedCABundle** to **Removed**; deleting the DSC Initialization does not delete the ConfigMaps.

To remove a CA bundle from a single namespace only, see [Removing the CA bundle from a single namespace](#) (OpenShift AI Self-Managed) or [Removing the CA bundle from a single namespace](#) (OpenShift AI Self-Managed in a disconnected environment).

Prerequisites

- You have cluster administrator privileges for your OpenShift cluster.
- You have installed the OpenShift CLI (**oc**) as described in the appropriate documentation for your cluster:
 - [Installing the OpenShift CLI](#) for OpenShift Container Platform
 - [Installing the OpenShift CLI](#) for Red Hat OpenShift Service on AWS

Procedure

1. In the OpenShift web console, click **Operators** → **Installed Operators** and then click the Red Hat OpenShift AI Operator.
2. Click the **DSC Initialization** tab.
3. Click the **default-dsci** object.
4. Click the **YAML** tab.
5. In the **spec** section, change the value of the **managementState** field for **trustedCABundle** to **Removed**:

```
spec:
  trustedCABundle:
    managementState: Removed
```

6. Click **Save**.

Verification

- Run the following command to verify that the **odh-trusted-ca-bundle** ConfigMap has been removed from all namespaces:

```
oc get configmaps --all-namespaces | grep odh-trusted-ca-bundle
```

The command should not return any ConfigMaps.

10.7.2. Removing the CA bundle from a single namespace

You can remove a custom Certificate Authority (CA) bundle from individual namespaces in OpenShift AI. This process disables the creation of the **odh-trusted-ca-bundle** configuration file (ConfigMap) for the specified namespace only.

To remove a CA bundle from all namespaces, [Removing the CA bundle from all namespaces](#) (OpenShift AI Self-Managed) or [Removing the CA bundle from all namespaces](#) (OpenShift AI Self-Managed in a disconnected environment).

Prerequisites

- You have cluster administrator privileges for your OpenShift cluster.
- You have installed the OpenShift CLI (**oc**) as described in the appropriate documentation for your cluster:
 - [Installing the OpenShift CLI](#) for OpenShift Container Platform
 - [Installing the OpenShift CLI](#) for Red Hat OpenShift Service on AWS

Procedure

- Run the following command to remove a CA bundle from a namespace. In the following command, *example-namespace* is the non-reserved namespace.

```
oc annotate ns example-namespace security.opendatahub.io/inject-trusted-ca-bundle=false
```

Verification

- Run the following command to verify that the CA bundle has been removed from the namespace. In the following command, *example-namespace* is the non-reserved namespace.

```
oc get configmap odh-trusted-ca-bundle -n example-namespace
```

The command should return **configmaps "odh-trusted-ca-bundle" not found**.

CHAPTER 11. VIEWING LOGS AND AUDIT RECORDS

As a cluster administrator, you can use the OpenShift AI Operator logger to monitor and troubleshoot issues. You can also use OpenShift audit records to review a history of changes made to the OpenShift AI Operator configuration.

11.1. CONFIGURING THE OPENSIFT AI OPERATOR LOGGER

You can change the log level for OpenShift AI Operator components by setting the **.spec.devFlags.logmode** flag for the **DSC Initialization/DSCI** custom resource during runtime. If you do not set a **logmode** value, the logger uses the INFO log level by default.

The log level that you set with **.spec.devFlags.logmode** applies to all components, not just those in a **Managed** state.

The following table shows the available log levels:

Log level	Stacktrace level	Verbosity	Output	Timestamp type
devel or development	WARN	INFO	Console	Epoch timestamps
"" (or no logmode value set)	ERROR	INFO	JSON	Human-readable timestamps
prod or production	ERROR	INFO	JSON	Human-readable timestamps

Logs that are set to **devel** or **development** generate in a plain text console format. Logs that are set to **prod**, **production**, or which do not have a level set generate in a JSON format.

Prerequisites

- You have administrator access to the **DSCInitialization** resources in the OpenShift cluster.
- You have installed the OpenShift CLI (**oc**) as described in the appropriate documentation for your cluster:
 - [Installing the OpenShift CLI](#) for OpenShift Container Platform
 - [Installing the OpenShift CLI](#) for Red Hat OpenShift Service on AWS

Procedure

1. Log in to the OpenShift as a cluster administrator.
2. Click **Operators** → **Installed Operators** and then click the Red Hat OpenShift AI Operator.
3. Click the **DSC Initialization** tab.
4. Click the **default-dsci** object.

- Click the **YAML** tab.
- In the **spec** section, update the **.spec.devFlags.logmode** flag with the log level that you want to set.

```
apiVersion: dscinitialization.opendatahub.io/v1
kind: DSCInitialization
metadata:
  name: default-dsci
spec:
  devFlags:
    logmode: development
```

- Click **Save**.

You can also configure the log level from the OpenShift CLI (**oc**) by using the following command with the **logmode** value set to the log level that you want.

```
oc patch dsci default-dsci -p '{"spec":{"devFlags":{"logmode":"development"}}}' --type=merge
```

Verification

- If you set the component log level to **devel** or **development**, logs generate more frequently and include logs at **WARN** level and above.
- If you set the component log level to **prod** or **production**, or do not set a log level, logs generate less frequently and include logs at **ERROR** level or above.

11.1.1. Viewing the OpenShift AI Operator logs

- Log in to the OpenShift CLI (**oc**).
- Run the following command to stream logs from all Operator pods:

```
for pod in $(oc get pods -l name=rhods-operator -n redhat-ods-operator -o name); do
  oc logs -f "$pod" -n redhat-ods-operator &
done
```

The Operator pod logs open in your terminal.

TIP

Press **Ctrl+C** to stop viewing. To fully stop all log streams, run **kill \$(jobs -p)**.

You can also view each Operator pod log in the OpenShift console by navigating to **Workloads → Pods**, selecting the **redhat-ods-operator** project, clicking a pod name, and then clicking the **Logs** tab.

11.2. VIEWING AUDIT RECORDS

Cluster administrators can use OpenShift auditing to see changes made to the OpenShift AI Operator configuration by reviewing modifications to the DataScienceCluster (DSC) and DSCInitialization (DSCI) custom resources. Audit logging is enabled by default in standard OpenShift cluster configurations. For

more information, see [Viewing audit logs](#) in the OpenShift documentation.



NOTE

In Red Hat OpenShift Service on AWS, audit logging is disabled by default because the Elasticsearch log store does not provide secure storage for audit logs. To configure log forwarding, see [Logging](#) in the Red Hat OpenShift Service on AWS documentation.

The following example shows how to use the OpenShift audit logs to see the history of changes made (by users) to the DSC and DSCI custom resources.

Prerequisites

- You have cluster administrator privileges for your OpenShift cluster.
- You have installed the OpenShift CLI (**oc**) as described in the appropriate documentation for your cluster:
 - [Installing the OpenShift CLI](#) for OpenShift Container Platform
 - [Installing the OpenShift CLI](#) for Red Hat OpenShift Service on AWS

Procedure

1. In a terminal window, if you are not already logged in to your OpenShift cluster as a cluster administrator, log in to the OpenShift CLI as shown in the following example:

```
$ oc login <openshift_cluster_url> -u <admin_username> -p <password>
```

2. To access the full content of the changed custom resources, set the OpenShift audit log policy to **WriteRequestBodies** or a more comprehensive profile. For more information, see [Configuring the audit log policy](#).
3. Fetch the audit log files that are available for the relevant control plane nodes. For example:

```
oc adm node-logs --role=master --path=kube-apiserver/ \
| awk '{ print $1 }' | sort -u \
| while read node ; do
  oc adm node-logs $node --path=kube-apiserver/audit.log < /dev/null
done \
| grep opendatahub > /tmp/kube-apiserver-audit-opendatahub.log
```

4. Search the files for the DSC and DSCI custom resources. For example:

```
jq 'select((.objectRef.apiGroup == "dscinitialization.opendatahub.io"
or .objectRef.apiGroup == "datasciencecluster.opendatahub.io")
and .user.username != "system:serviceaccount:redhat-ods-operator:redhat-ods-
operator-controller-manager"
and .verb != "get" and .verb != "watch" and .verb != "list")' < /tmp/kube-apiserver-
audit-opendatahub.log
```

Verification

- The commands return relevant log entries.

TIP

To configure the log retention time, see the [Logging](#) section in the OpenShift documentation.

CHAPTER 12. TROUBLESHOOTING COMMON INSTALLATION PROBLEMS

If you are experiencing difficulties installing the Red Hat OpenShift AI Operator, read this section to understand what could be causing the problem and how to resolve it.

If the problem is not included here or in the release notes, contact [Red Hat Support](#). When opening a support case, it is helpful to include debugging information about your cluster. You can collect this information by using the **must-gather** tool as described in [Must-Gather for Red Hat OpenShift AI](#) and [Gathering data about your cluster](#).

You can also adjust the log level of OpenShift AI Operator components to increase or reduce log verbosity to suit your use case. For more information, see [Configuring the OpenShift AI Operator logger](#).

12.1. THE RED HAT OPENSIFT AI OPERATOR CANNOT BE RETRIEVED FROM THE IMAGE REGISTRY

Problem

When attempting to retrieve the Red Hat OpenShift AI Operator from the image registry, an **Failure to pull from quay** error message appears. The Red Hat OpenShift AI Operator might be unavailable for retrieval in the following circumstances:

- The image registry is unavailable.
- There is a problem with your network connection.
- Your cluster is not operational and is therefore unable to retrieve the image registry.

Diagnosis

Check the logs in the **Events** section in OpenShift for further information about the **Failure to pull from quay** error message.

Resolution

- Contact Red Hat support.

12.2. OPENSIFT AI DOES NOT INSTALL ON UNSUPPORTED INFRASTRUCTURE

Problem

You are deploying on an environment that is not documented as supported by the Red Hat OpenShift AI Operator.

Diagnosis

1. In the OpenShift web console, switch to the **Administrator** perspective.
2. Click **Workloads → Pods**.
3. Set the **Project** to **All Projects** or **redhat-ods-operator**.

4. Click the **rhods-operator-`<random string>`** pod that shows an error in the **Status** column. The **Pod details** page appears.
5. Click **Logs**.
6. Select **rhods-operator** from the drop-down list.
7. Check the log for the **ERROR: Deploying on \$infrastructure, which is not supported. Failing Installation** error message.

Resolution

- Before proceeding with a new installation, ensure that you have a fully supported environment on which to install OpenShift AI. For more information, see [Red Hat OpenShift AI: Supported Configurations](#).

12.3. THE CREATION OF THE OPENSIFT AI CUSTOM RESOURCE (CR) FAILS

Problem

During the installation process, the OpenShift AI Custom Resource (CR) does not get created. This issue occurs in unknown circumstances.

Diagnosis

1. In the OpenShift web console, switch to the **Administrator** perspective.
2. Click **Workloads → Pods**.
3. Set the **Project** to **All Projects** or **redhat-ods-operator**.
4. Click the **rhods-operator-`<random string>`** pod that shows an error in the **Status** column. The **Pod details** page appears.
5. Click **Logs**.
6. Select **rhods-operator** from the drop-down list.
7. Check the log for the **ERROR: Attempt to create the ODH CR failed.** error message.

Resolution

- Contact Red Hat support.

12.4. THE CREATION OF THE OPENSIFT AI NOTEBOOKS CUSTOM RESOURCE (CR) FAILS

Problem

During the installation process, the OpenShift AI Notebooks Custom Resource (CR) does not get created. This issue occurs in unknown circumstances.

Diagnosis

1. In the OpenShift web console, switch to the **Administrator** perspective.
2. Click **Workloads → Pods**.
3. Set the **Project** to **All Projects** or **redhat-ods-operator**.
4. Click the **rhods-operator-*<random string>*** pod that shows an error in the **Status** column.
The **Pod details** page appears.
5. Click **Logs**.
6. Select **rhods-operator** from the drop-down list.
7. Check the log for the **ERROR: Attempt to create the RHODS Notebooks CR failed.** error message.

Resolution

- Contact Red Hat support.

12.5. THE OPENSIFT AI DASHBOARD IS NOT ACCESSIBLE

Problem

After installing OpenShift AI, the **redhat-ods-applications**, **redhat-ods-monitoring**, and **redhat-ods-operator** project namespaces are **Active** but you cannot access the dashboard due to an error in one of the pods.

Diagnosis

1. In the OpenShift web console, switch to the **Administrator** perspective.
2. Click **Workloads → Pods**.
3. Set the **Project** to **All Projects**.
4. Click **Filter** and select the checkbox for every status except **Running** and **Completed**.
The page displays the pods that have an error.

Resolution

- To see more information and troubleshooting steps for a pod, on the **Pods** page, click the link in the **Status** column for the pod.
- If the **Status** column does not display a link, click the pod name to open the pod details page and then click the **Logs** tab.

12.6. REINSTALLING OPENSIFT AI FAILS WITH AN ERROR

Problem

After uninstalling the OpenShift AI Operator and reinstalling it by using the CLI, the reinstallation fails with an **unable to find DSCInitialization** error in one of the OpenShift AI Operator pod logs. This issue can occur if the **Auth** custom resource from the previous installation was not deleted after uninstalling

the OpenShift AI Operator and before reinstalling it. For more information, see [Understanding the uninstallation process](#).

Diagnosis

1. In the OpenShift web console, switch to the **Administrator** perspective.
2. Click **Workloads** → **Pods**.
3. Set the **Project** to **All Projects** or **redhat-ods-operator**.
4. Click the **rhods-operator-<random string>** pod that shows an error in the **Status** column. The **Pod details** page appears.
5. Click **Logs**.
6. Select **rhods-operator** from the drop-down list.
7. Check the log for an error message similar to the following:

```
{ "name": "auth", "namespace": "", "name": "auth", "reconcileID": "7bff53ae-1252-46fe-831a-fdc824078a1b", "error": "unable to find DSCInitialization", "stacktrace": "sigs.k8s.io/controller-runtime/pkg/internal/controller." }
```

Resolution

1. Uninstall the OpenShift AI Operator.
2. Delete the **Auth** custom resource:
 - a. In the OpenShift web console, switch to the **Administrator** perspective.
 - b. Click **API Explorer**.
 - c. From the **All groups** drop-down list, select or enter **services.platform.opendatahub.io**.
 - d. Click the **Auth** kind.
 - e. Click the **Instances** tab.
 - f. Click the action menu (⋮) and select **Delete Auth**. The **Delete Auth** dialog appears.
 - g. Click **Delete**.
3. Install the OpenShift AI Operator again.

12.7. THE DEDICATED-ADMINS ROLE-BASED ACCESS CONTROL (RBAC) POLICY CANNOT BE CREATED

Problem

The Role-based access control (RBAC) policy for the dedicated-admins group in the target project cannot be created. This issue occurs in unknown circumstances.

Diagnosis

Diagnosis

1. In the OpenShift web console, switch to the **Administrator** perspective.
2. Click **Workloads → Pods**.
3. Set the **Project** to **All Projects** or **redhat-ods-operator**.
4. Click the **rhods-operator-`<random string>`** pod that shows an error in the **Status** column.
The **Pod details** page appears.
5. Click **Logs**.
6. Select **rhods-operator** from the drop-down list.
7. Check the log for the **ERROR: Attempt to create the RBAC policy for dedicated admins group in \$target_project failed.** error message.

Resolution

- Contact Red Hat support.

12.8. THE PAGERDUTY SECRET DOES NOT GET CREATED

Problem

An issue with Managed Tenants SRE automation process causes the PagerDuty's secret to not get created.

Diagnosis

1. In the OpenShift web console, switch to the **Administrator** perspective.
2. Click **Workloads → Pods**.
3. Set the **Project** to **All Projects** or **redhat-ods-operator**.
4. Click the **rhods-operator-`<random string>`** pod that shows an error in the **Status** column.
The **Pod details** page appears.
5. Click **Logs**.
6. Select **rhods-operator** from the drop-down list.
7. Check the log for the **ERROR: Pagerduty secret does not exist** error message.

Resolution

- Contact Red Hat support.

12.9. THE SMTP SECRET DOES NOT EXIST

Problem

An issue with Managed Tenants SRE automation process causes the SMTP secret to not get created.

Diagnosis

1. In the OpenShift web console, switch to the **Administrator** perspective.
2. Click **Workloads → Pods**.
3. Set the **Project** to **All Projects** or **redhat-ods-operator**.
4. Click the **rhods-operator-<random string>** pod that shows an error in the **Status** column.
The **Pod details** page appears.
5. Click **Logs**.
6. Select **rhods-operator** from the drop-down list.
7. Check the log for the **ERROR: SMTP secret does not exist** error message.

Resolution

- Contact Red Hat support.

12.10. THE ODH PARAMETER SECRET DOES NOT GET CREATED

Problem

An issue with the OpenShift AI Operator's flow could result in failure to create the ODH parameter.

Diagnosis

1. In the OpenShift web console, switch to the **Administrator** perspective.
2. Click **Workloads → Pods**.
3. Set the **Project** to **All Projects** or **redhat-ods-operator**.
4. Click the **rhods-operator-<random string>** pod that shows an error in the **Status** column.
The **Pod details** page appears.
5. Click **Logs**.
6. Select **rhods-operator** from the drop-down list.
7. Check the log for the **ERROR: Addon managed odh parameter secret does not exist.** error message.

Resolution

- Contact Red Hat support.

CHAPTER 13. UNINSTALLING RED HAT OPENSIFT AI SELF-MANAGED

This section shows how to use the OpenShift CLI (**oc**) to uninstall the Red Hat OpenShift AI Operator and any OpenShift AI components installed and managed by the Operator.



NOTE

Using the OpenShift CLI (**oc**) is the recommended way to uninstall the Operator. Depending on your version of OpenShift, using the web console to perform the uninstallation might not prompt you to uninstall all associated components. This could leave you unclear about the final state of your cluster.

13.1. UNDERSTANDING THE UNINSTALLATION PROCESS

Installing Red Hat OpenShift AI created several custom resource instances on your OpenShift cluster for various components of OpenShift AI. After installation, users likely created several additional resources while using OpenShift AI. Uninstalling OpenShift AI removes the resources that were created by the Operator, but retains the resources created by users to prevent inadvertently deleting information you might want.

What is deleted

Uninstalling OpenShift AI removes the following resources from your OpenShift cluster:

- **DataScienceCluster** custom resource instance and the custom resource instances it created for each component
- **DSCInitialization** custom resource instance
- **Auth** custom resource instance created during or after installation
- **FeatureTracker** custom resource instances created during or after installation
- **ServiceMesh** custom resource instance created by the Operator during or after installation
- **KNativeServing** custom resource instance created by the Operator during or after installation
- **redhat-ods-applications**, **redhat-ods-monitoring**, and **rhods-notebooks** namespaces created by the Operator
- Workloads in the **rhods-notebooks** namespace
- **Subscription**, **ClusterServiceVersion**, and **InstallPlan** objects
- **KfDef** object (version 1 Operator only)

What might remain

Uninstalling OpenShift AI retains the following resources in your OpenShift cluster:

- Data science projects created by users
- Custom resource instances created by users

- Custom resource definitions (CRDs) created by users or by the Operator

While these resources might still remain in your OpenShift cluster, they are not functional. After uninstalling, Red Hat recommends that you review the data science projects and custom resources in your OpenShift cluster and delete anything no longer in use to prevent potential issues, such as pipelines that cannot run, notebooks that cannot be undeployed, or models that cannot be undeployed.

Additional resources

[Operator Lifecycle Manager \(OLM\) uninstall documentation](#)

13.2. UNINSTALLING OPENSIFT AI SELF-MANAGED BY USING THE CLI

The following procedure shows how to use the OpenShift CLI (**oc**) to uninstall the Red Hat OpenShift AI Operator and any OpenShift AI components installed and managed by the Operator.

Prerequisites

- You have cluster administrator privileges for your OpenShift cluster.
- You have installed the OpenShift CLI (**oc**) as described in the appropriate documentation for your cluster:
 - [Installing the OpenShift CLI](#) for OpenShift Container Platform
 - [Installing the OpenShift CLI](#) for Red Hat OpenShift Service on AWS
- You have backed up the persistent disks or volumes used by your persistent volume claims (PVCs).

Procedure

1. Open a new terminal window.
2. Log in to your OpenShift cluster as a cluster administrator, as shown in the following example:

```
$ oc login <openshift_cluster_url> -u system:admin
```

3. Create a **ConfigMap** object for deletion of the Red Hat OpenShift AI Operator.

```
$ oc create configmap delete-self-managed-odh -n redhat-ods-operator
```

4. To delete the **rhods-operator**, set the **addon-managed-odh-delete** label to **true**.

```
$ oc label configmap/delete-self-managed-odh api.openshift.com/addon-managed-odh-delete=true -n redhat-ods-operator
```

5. When all objects associated with the Operator are removed, delete the **redhat-ods-operator** project.

- a. Set an environment variable for the **redhat-ods-applications** project.

```
$ PROJECT_NAME=redhat-ods-applications
```

- b. Wait until the **redhat-ods-applications** project has been deleted.

```
while oc get project $PROJECT_NAME &> /dev/null; do
  echo "The $PROJECT_NAME project still exists"
  sleep 1
done
echo "The $PROJECT_NAME project no longer exists"
```

When the **redhat-ods-applications** project has been deleted, you see the following output.

```
The redhat-ods-applications project no longer exists
```

- c. When the **redhat-ods-applications** project has been deleted, delete the **redhat-ods-operator** project.

```
$ oc delete namespace redhat-ods-operator
```

Verification

1. Confirm that the **rhods-operator** subscription no longer exists.

```
$ oc get subscriptions --all-namespaces | grep rhods-operator
```

2. Confirm that the following projects no longer exist.

- **redhat-ods-applications**
- **redhat-ods-monitoring**
- **redhat-ods-operator**
- **rhods-notebooks**

```
$ oc get namespaces | grep -e redhat-ods* -e rhods*
```

The **rhods-notebooks** project existed only if you installed the workbenches component of OpenShift AI. See [Installing and managing Red Hat OpenShift AI components](#).